UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
INSTITUTO DE COMPUTAÇÃO
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

DIEGO VASCONCELOS SCHARDOSIM DE MATOS

Animação Física Simplificada em Web

RIO DE JANEIRO
2026

DIEGO VASCONCELOS SCHARDOSIM DE MATOS

Animação Física Simplificada em Web

Trabalho de conclusão de curso de graduação apresentado ao Instituto de Computação da Universidade Federal do Rio de Janeiro como parte dos requisitos para obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Prof. Cláudio Esperança

RIO DE JANEIRO

2026

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema de animação física simplificada para aplicações Web, fundamentado no método de Jakobsen (2001). O objetivo é investigar e demonstrar como técnicas de simulação leve podem produzir movimentos coerentes, estáveis e visualmente naturais, mesmo em ambientes com recursos computacionais limitados, como navegadores modernos. Para isso, são integradas abordagens clássicas de detecção e resposta a colisões, incluindo estratégias de Filtragem e Refinamento, bem como métodos geométricos amplamente utilizados em sistemas interativos. Além disso, o projeto incorpora otimizações estruturais, como subdivisão espacial e processamento em thread separada, buscando garantir escalabilidade e desempenho em cenários com múltiplos objetos dinâmicos. Os resultados obtidos evidenciam que é possível alcançar simulações eficientes e responsivas mantendo baixo custo computacional, tornando essas técnicas adequadas para jogos, visualizações interativas e aplicações educacionais na Web.

Palavras-chave: Computação gráfica; Animação Física; Detecção de Colisão; Resposta a Colisão; Corpos Rígidos e Deformáveis; Web.

ABSTRACT

This work presents the development of a simplified physical animation system for Web applications, based on Jakobsen (2001)'s method. The goal is to investigate and demonstrate how lightweight simulation techniques can produce coherent, stable, and visually natural motion, even in environments with limited computational resources, such as modern browsers. To achieve this, classical approaches to collision detection and response are integrated, including filtering and refining strategies, as well as geometric methods widely used in interactive systems. In addition, the project incorporates structural optimizations such as spatial subdivision and processing in a separate thread, aiming to ensure scalability and performance in scenarios with multiple dynamic objects. The results obtained show that it is possible to achieve efficient and responsive simulations while maintaining low computational cost, making these techniques suitable for games, interactive visualizations, and educational applications on the Web.

Keywords: Computer Graphics; Physics Animation; Collision Detection; Collision Response; Rigid and Deformable Bodies; Web.

LISTA DE ILUSTRAÇÕES

Figura 1 – A distância atual entre as partículas p_1 e p_2 está inválida e deve ser corrigida por projeção. 14
Figura 2 – A articulação é formada simplesmente fazendo os dois corpos compartilharem a mesma partícula 15
Figura 3 – Comparação entre polígono convexo (à esquerda) e polígono côncavo (à direita). 16
Figura 4 – Aresta escolhida em amarelo, eixo separador perpendicular à face. Fonte: Autor 19
Figura 5 – Fecho convexo de um conjunto de pontos. 21
Figura 6 – Soma de Minkowski, entre os conjuntos A e B. 21
Figura 7 – A função suporte na forma implícita $A - B$ ao longo da direção $\vec{d}$. . . . 23
Figura 8 – Da esquerda para direita 0-simplex, 1-simplex, 2-simplex e 3-simplex . 23
Figura 9 – Ilustração do teorema de Carathéodory. O ponto $\vec{x}$ está no interior do fecho convexo de P, e também está no interior do fecho convexo de P'. 24
Figura 10 – Divisão de regiões para o caso 1-simplex. 26
Figura 11 – Regiões para o caso 2-simplex. 28
Figura 12 – Processo de separação caso colisão vértice-aresta. Correção das posição para configuração válida. Perceba que, como p está mais próximo de x_1, esse vértice é movido mais. 34
Figura 13 – Uma sequência de iterações do algoritmo de expansão de politopos. Uma seta indica o vetor mais curto $\vec{p}$. Um ponto aberto indica o novo vértice $\vec{w}$. Fonte: (BERGEN, 2001) 36
Figura 14 – Em (A) os objetos delimitadores não se intersectam, em (B) eles se intersectam porém os objetos não se intersectam, em (C) eles se intersectam e os objetos se intersectam. 39
Figura 15 – Da esquerda para direita esfera delimitadora, caixa delimitadora alinhada aos eixos, caixa delimitadora orientada, K-DOP e fecho convexo. 39
Figura 16 – Etapas do quickhull fonte: (ZENG; ZHONG; PAN, 2024). 42
Figura 17 – Divisão espacial em grade uniforme 43
Figura 18 – À esquerda esquema tradicional single-thread. À direita esquema com simulação rodando numa thread dedicada a passo fixo. 47
Figura 19 – Ambiente de simulação com múltiplos objetos convexos em uma caixa, permitindo a configuração de parâmetros de colisão e filtragem. Fonte: Autor. 51
Figura 20 – Simulação de estruturas deformáveis, como tecidos e treliças, utilizando restrições entre partículas e detecção de colisão. Fonte: Autor. 52

Figura 21 – Número médio de testes de colisão em função do número de objetos para diferentes configurações: SAT e GJK/EPA, com e sem utilização de grade uniforme. Fonte: Autor. 54
Figura 22 – Tempo médio de execução em função do número de objetos para diferentes configurações: SAT e GJK/EPA, com e sem utilização de grade uniforme. Fonte: Autor. 55
Figura 23 – Tempo médio de execução em função do número de objetos com a biblioteca *Planck.js*. Fonte: Autor. 55

SUMÁRIO

1 INTRODUÇÃO 7

2 ANIMAÇÃO BASEADA EM FÍSICA 10
2.1 CONCEITOS E DEFINIÇÕES 10
2.2 DINÂMICA DE PARTÍCULAS 11
2.3 REPRESENTAÇÃO DE CORPOS RÍGIDOS 11
2.4 SIMULAÇÃO DE CORPOS DEFORMÁVEIS 12
2.5 MÉTODOS NUMÉRICOS EM SIMULAÇÃO 13
2.6 RESTRIÇÕES GEOMÉTRICAS 14
2.7 RESOLVENDO RESTRIÇÕES CONCORRENTES POR RELAXAMENTO 15
2.8 POLÍGONOS, POLIEDROS E POLITOPOS 16

3 DETECÇÃO DE COLISÕES 18
3.1 TEOREMA DO EIXO SEPARADOR (SAT) 18
3.2 ALGORITMO DE GILBERT-JOHNSON-KEERTHI (GJK) 20

4 RESPOSTA A COLISÕES 32
4.1 MÉTODOS DINÂMICOS NA SIMULAÇÃO FÍSICA 32
4.2 PROCESSO DE SEPARAÇÃO 33
4.3 SAT COM CÁLCULO DO MTV 34
4.4 ALGORITMO DE EXPANSÃO DE POLITOPOS (EPA) 35
4.5 LIMITAÇÕES 36

5 OTIMIZAÇÕES 38
5.1 OBJETOS DELIMITADORES 38
5.2 FILTRAGEM E REFINAMENTO 41
5.3 COMO DETERMINAR O INTERVALO DE INTEGRAÇÃO DA FÍSICA 45
5.4 EXECUÇÃO PARALELA E PARALELISMO DE DADOS NA SIMULAÇÃO FÍSICA 46

6 IMPLEMENTAÇÃO E AVALIAÇÃO EXPERIMENTAL . . . 50
6.1 ARQUITETURA E IMPLEMENTAÇÃO 50
6.2 AMBIENTE DE VISUALIZAÇÃO 51
6.3 METODOLOGIA DE TESTES 51
6.4 RESULTADOS EXPERIMENTAIS 53

7 CONCLUSÕES 56

REFERÊNCIAS 57

1 INTRODUÇÃO

O campo da Computação Gráfica evoluiu significativamente nas últimas décadas, impulsionado pelo aumento da capacidade computacional e pela especialização do hardware gráfico. Esta área investiga métodos e algoritmos para gerar, manipular e representar imagens digitais de forma eficiente, desempenhando um papel central em aplicações de design, engenharia, entretenimento e visualização científica. Como destaca Azevedo (2003), a computação gráfica combina matemática e arte, oferecendo meios para representar fenômenos complexos e criar imagens inviáveis pelos métodos tradicionais.

Em paralelo, o avanço dos programas interativos, especialmente jogos eletrônicos e simulações físicas em tempo real, intensificou a necessidade de técnicas capazes de combinar realismo visual com desempenho computacional. De acordo com Möller, Haines e Hoffman (2018) num programa interativo, uma imagem é exibida, o usuário reage, e essa reação influencia as próximas imagens a serem geradas. Esse ciclo de interação deve ocorrer a uma taxa suficientemente alta para que o usuário não veja imagens individuais, mas sim se sinta imerso em um processo dinâmico. A taxa na qual as imagens são exibidas é medida em quadros por segundo (FPS) ou Hertz (Hz). Para aplicações interativas, requer-se ao menos uma taxa de 6 FPS sendo que taxas mais elevadas tornam a experiência mais imersiva.

No contexto das aplicações interativas, destacam-se aquelas que realizam animação, isto é, a produção de imagens dinâmicas que representam objetos em movimento. Em geral, essas animações podem ser classificadas em dois grandes grupos: (i) animações *keyframe*, nas quais artistas definem manualmente os quadros-chave e as interpolações; e (ii) animações baseadas em física, em que os movimentos emergem da simulação de leis físicas, permitindo comportamentos naturais e interações complexas entre objetos.

O presente trabalho aborda a implementação de animação física 2D na Web. A escolha deste tema justifica-se por sua ampla aplicabilidade em jogos digitais, simuladores educacionais e interfaces visuais interativas. A popularidade do problema decorre de sua relevância prática e de seus desafios teóricos, que envolvem tanto a modelagem geométrica dos objetos quanto sua evolução temporal segundo princípios físicos.

Diversos jogos e motores gráficos modernos adotam animações físicas para produzir experiências imersivas e responsivas. Exemplos incluem sistemas de queda, *ragdolls*, tecidos, corpos rígidos e fluidos. Em todos esses casos, a simulação envolve um conjunto de corpos que possuem propriedades físicas, como massa, forma, velocidade e aceleração, e cujas interações são determinadas por colisões, forças e restrições. Uma animação física une dois domínios: a geometria, necessária para detectar e calcular contatos, e a física, responsável pela evolução dinâmica dos corpos.

De forma geral, um sistema de animação física segue o seguinte esquema: (i) definição

dos corpos e de suas propriedades físicas; (ii) integração numérica para atualizar suas posições e velocidades; (iii) detecção de colisões, que identifica se e como os objetos se interceptam; (iv) resposta às colisões, que corrige interpenetrações e determina novas velocidades; e, opcionalmente, (v) renderização dos resultados ao usuário. Por sua natureza iterativa e dependente de múltiplos módulos, trata-se de um problema complexo, sensível ao desempenho e à precisão.

A relevância dessa classe de problemas motivou o desenvolvimento de diversos motores de física que alavancam hardware gráfico para melhorar o desempenho. Entre os quais podemos citar motores físicos comerciais e de código aberto como o Havok (Havok, 2024) popularizou o uso profissional de física em jogos de grande porte, oferecendo um sistema robusto de colisões e restrições. O NVIDIA PhysX (NVIDIA Omniverse,) introduziu aceleração por GPU, permitindo simulações mais ricas. Motores como Bullet Physics (COUMANS, 2015) e Box2D (CATTO, 2011), ambos de código aberto, democratizaram o acesso a ferramentas de alta qualidade, tornando-se amplamente adotados em pesquisas, jogos independentes e aplicações embarcadas. Tais ferramentas demonstram a importância do tema e como algoritmos otimizados, aliados a hardware moderno, permitem simulações estáveis em tempo real.

Jakobsen (JAKOBSEN, 2001) propôs durante o desenvolvimento do jogo *Hitman: Codename 47* um esquema simplificado de simulação física que é capaz de modelar corpos rígidos e deformáveis, e tecidos, sem computar explicitamente matrizes de orientação, torques ou tensores de inércia. Seu método combina manutenção iterativa de restrições, resposta a colisões por projeção e uso de aproximações eficientes. Entretanto, o método omite diversos detalhes importantes, como estratégias de detecção de colisões, tratamento de um grande número de restrições e mecanismos de otimização para múltiplos objetos, lacunas que este trabalho busca explorar.

Independentemente da complexidade da simulação física, toda aplicação interativa requer um módulo final de renderização, responsável por apresentar ao usuário o estado atualizado da cena. No contexto Web, tecnologias como WebGL permitem que essa etapa seja realizada com aceleração gráfica, integrando o fluxo completo de animação física em tempo real.

Neste trabalho, o objetivo é desenvolver um protótipo inspirado no método de Jakobsen, apresentando soluções para detecção e resposta a colisões com foco em aplicações Web. São metas específicas:

- Revisar os principais conceitos de animação baseada em física e integração numérica;
- Implementar algoritmos de detecção e resposta a colisão;
- Desenvolver uma simulação física baseada em partículas e restrições utilizando o método de Jakobsen;

- Aplicar técnicas de otimização e processamento multi-threaded;
- Avaliar o desempenho e a estabilidade do sistema em diferentes cenários.

O trabalho está organizado da seguinte forma: O Capítulo 2 fundamenta a animação baseada em física e detalha o método de Jakobsen. O Capítulo 3 descreve os algoritmos essenciais para a detecção de colisões. O Capítulo 4 aborda as técnicas de resposta a colisão, incluindo projeção de posição. O Capítulo 5 foca nas estratégias de otimização para tempo real, filtragem e refinamento, objetos delimitadores e processamento em thread separada. O Capítulo 6 apresenta a metodologia experimental e a discussão dos resultados. Por fim, o Capítulo 7 resume as contribuições, discute limitações e propõe trabalhos futuros.

2 ANIMAÇÃO BASEADA EM FÍSICA

A animação baseada em física é uma abordagem de geração de movimento que aparenta seguir princípios físicos básicos, mesmo que as equações envolvidas sejam tratadas de forma aproximada ou altamente simplificada. Diferentemente da animação tradicional, na qual o animador define manualmente posições e rotações ao longo do tempo (do inglês, *keyframe animation*), a animação física permite que o comportamento dos objetos emerja naturalmente das forças, restrições e interações entre os corpos simulados.

Nesse contexto, a animação física simplificada busca um equilíbrio entre precisão e desempenho: modelos matemáticos são utilizados como guia, mas a prioridade está na estabilidade visual e na resposta interativa. Diferentemente da simulação científica, onde precisão e correção numérica são cruciais, na animação para fins gráficos ou interativos o objetivo não é obter resultados fisicamente corretos, mas sim verossimilhança visual (plausibilidade). O foco está em transmitir sensação de peso, inércia e colisões de maneira convincente ao usuário, mesmo quando obtidos por heurísticas.

2.1 CONCEITOS E DEFINIÇÕES

O objetivo central da animação baseada em física é resolver numericamente as equações que descrevem o movimento de objetos em um mundo virtual. Tais equações derivam das leis fundamentais da mecânica clássica, formuladas por Isaac Newton.

> **Primeira lei de Newton (Inércia):** Na ausência de forças externas, um objeto em repouso permanece em repouso e um objeto em movimento continua em movimento com velocidade constante. Apenas forças externas podem alterar o estado de movimento.
>
> **Segunda lei de Newton (Princípio Fundamental da Dinâmica):** Para um corpo de massa constante m submetido a uma força $\vec{F}$, o movimento é descrito por:
> $$\vec{F} = m\vec{a} = m\frac{d\vec{v}}{dt} = m\frac{d^2\vec{x}}{dt^2},$$
> onde $\vec{x}$ é a posição do corpo, $\vec{v}$ é sua velocidade e t é o tempo.
>
> **Terceira lei de Newton (Ação e Reação):** Para toda força exercida em um corpo existe uma força de igual magnitude e direção oposta exercida no corpo que a gerou.

Em implementações simplificadas, efeitos complexos como atrito, restituição e deformação são aproximados por modelos empíricos. De maneira geral, o ciclo de uma simulação física engloba:

1. coleta e soma das forças aplicadas (gravidade, vento, atrito etc.);
2. integração temporal das equações de movimento;
3. detecção e resposta a colisões;
4. atualização das posições e posterior renderização.

2.2 DINÂMICA DE PARTÍCULAS

A dinâmica de partículas é uma abordagem na qual o sistema é composto por partículas independentes. Cada partícula possui posição, velocidade e massa, e suas interações são modeladas por forças (como gravidade ou molas) ou por restrições geométricas (como manter distâncias constantes).

O estado de uma particula no instante t é dado por:

$$X(t) = \begin{pmatrix} \vec{x}(t) \\ \vec{v}(t) \end{pmatrix}.$$

Seja $F(t)$ a soma das forças que atuam sobre a partícula e m sua massa. O movimento pode ser descrito por:

$$\frac{d}{dt}X(t) = \frac{d}{dt}\begin{pmatrix} \vec{x}(t) \\ \vec{v}(t) \end{pmatrix} = \begin{pmatrix} \vec{v}(t) \\ \dfrac{F(t)}{m} \end{pmatrix}.$$

Esta abordagem é flexível e serve como base para simular sistemas complexos, como tecidos, fluidos e corpos deformáveis, onde o comportamento macroscópico emerge da interação coletiva das partículas.

2.3 REPRESENTAÇÃO DE CORPOS RÍGIDOS

Um corpo rígido é um objeto cuja forma e volume permanecem invariáveis durante a simulação. Em termos matemáticos, a distância entre quaisquer dois pontos do corpo é constante, independentemente das forças aplicadas. Essa suposição simplifica o problema, permitindo representar o corpo apenas por grandezas globais: posição, orientação e velocidades linear e angular.

A representação matemática de um corpo rígido é dada por:

- **Posição** $\vec{p}$: coordenadas do centro de massa;
- **Orientação** R: matriz de rotação ou quatérnio;
- **Velocidade linear** $\vec{v}$: variação temporal da posição;
- **Velocidade angular** $\vec{\omega}$: variação temporal da orientação;
- **Massa** m e **tensor de inércia** I: medidas de resistência à aceleração.

2.4 SIMULAÇÃO DE CORPOS DEFORMÁVEIS

Enquanto a dinâmica de corpos rígidos assume que a distância entre os pontos constituintes do objeto permanece inalterada, a simulação de corpos deformáveis lida com objetos que alteram sua forma sob a ação de forças externas. Esta categoria abrange uma vasta gama de fenômenos físicos, desde o comportamento elástico de uma bola de borracha até a dinâmica complexa de tecidos, cabelos e fluidos.

Na mecânica do contínuo, a deformação é descrita pela mudança na configuração métrica do material, gerando tensões internas que tendem a restaurar o objeto ao seu estado de repouso ou dissipar energia na forma de deformação plástica. No contexto da computação gráfica em tempo real, no entanto, modelos discretos simplificados, tipicamente baseados em partículas, são preferidos em relação a métodos mais complexos, como por exemplo os Métodos de Elementos Finitos (FEM, do inglês *Finit Element Methods*).

Sistemas Massa-Mola

A abordagem mais comum para simular deformações em jogos e aplicações interativas é o modelo Massa-Mola. Neste modelo, um objeto é discretizado em um conjunto de partículas pontuais com massa m, conectadas por molas ideais sem massa.

A força interna exercida por uma mola entre duas partículas i e j é descrita pela Lei de Hooke:

$$\vec{F}_{elastica} = -k_s(|\vec{x}_i - \vec{x}_j| - L_0)\frac{\vec{x}_i - \vec{x}_j}{|\vec{x}_i - \vec{x}_j|},$$

onde k_s é a constante de rigidez, L_0 é o comprimento de repouso da mola e $\vec{x}$ são as posições das partículas. Para garantir a estabilidade numérica e simular a perda de energia, adiciona-se frequentemente um termo de amortecimento:

$$\vec{F}_{amortecimento} = -k_d(\vec{v}_i - \vec{v}_j)\frac{\vec{x}_i - \vec{x}_j}{|\vec{x}_i - \vec{x}_j|},$$

onde k_d é o coeficiente de amortecimento e $\vec{v}$ são as velocidades.

Estruturação Topológica para Tecidos

Para simular superfícies deformáveis, como tecidos, as partículas são organizadas em uma grade. A estabilidade e o comportamento visual do tecido dependem de como essas molas (ou restrições) são conectadas. Uma topologia robusta geralmente emprega três tipos de conexões:

1. Molas Estruturais: Conectam partículas vizinhas diretas (horizontal e verticalmente). Elas resistem à tração e compressão básicas, mantendo a integridade da malha.

2. Molas de Cisalhamento: Conectam partículas diagonalmente vizinhas. Sua função é impedir que o tecido se distorça ou "deslize" sobre si mesmo, mantendo a estabilidade dos quadriláteros da malha.

3. Molas de Flexão: Conectam partículas alternadas (pulando um vizinho). Elas resistem à dobra do tecido, impedindo que ele se comporte como uma malha infinitamente flexível e conferindo-lhe uma certa rigidez à curvatura.

2.5 MÉTODOS NUMÉRICOS EM SIMULAÇÃO

A simulação de movimento depende da solução numérica das equações diferenciais que descrevem a dinâmica dos corpos. Diversos métodos de integração podem ser utilizados, cada um com um equilíbrio distinto entre precisão, estabilidade e custo computacional.

Método de Euler

O método de Euler é o mais simples e intuitivo. Ele atualiza a posição e a velocidade de acordo com a aceleração atual:

$$\vec{v}_{t+\Delta t} = \vec{v}_t + \vec{a}_t \Delta t,$$
$$\vec{x}_{t+\Delta t} = \vec{x}_t + \vec{v}_t \Delta t.$$

Apesar de sua simplicidade, o método de Euler tende a ser numericamente instável, especialmente em sistemas oscilatórios (como molas), pois o erro de integração cresce rapidamente ao longo do tempo.

Método Semi-implícito de Euler

Uma variação estável do método de Euler consiste em atualizar primeiro a velocidade e depois a posição, utilizando a nova velocidade no cálculo:

$$\vec{v}_{t+\Delta t} = \vec{v}_t + \vec{a}_t \Delta t,$$
$$\vec{x}_{t+\Delta t} = \vec{x}_t + \vec{v}_{t+\Delta t} \Delta t.$$

Essa pequena modificação melhora a conservação de energia e reduz a instabilidade numérica, sendo amplamente adotado em motores como o Box2D.

Integração de Verlet

Verlet é um método de segunda ordem que não armazena explicitamente a velocidade. A nova posição é calculada com base na posição atual, na posição anterior e na aceleração:

$$\vec{x}_{t+\Delta t} = 2\vec{x}_t - \vec{x}_{t-\Delta t} + \vec{a}_t \Delta t^2.$$

A velocidade, quando necessária para cálculos de amortecimento ou jogabilidade, é derivada implicitamente:

$$\vec{v}_t \approx \frac{\vec{x}_{t+\Delta t} - \vec{x}_{t-\Delta t}}{2\Delta t}.$$

Esse é o método usado por Jakobsen (2001), por ser um método estável e eficiente, especialmente em sistemas sujeitos a restrições geométricas além de eliminar a necessidade de armazenar explicitamente a velocidade, utilizando as posições atual e anterior para estimar a nova posição.

2.6 RESTRIÇÕES GEOMÉTRICAS

Jakobsen (2001) descreve uma restrição geométrica simples para compor estruturas complexas, a restrição linear (ou restrição de distância) mantém uma distância fixa d entre duas partículas i e j. Dessa forma o conjunto de partículas deve satisfazer a todo instante uma coleção de equações na forma:

$$|\vec{x_i} - \vec{x_j}| = d. \tag{2.1}$$

Mesmo que as posições das partículas estejam inicialmente corretas, depois de um passo de simulação a distância entre elas pode se tornar inválida (devido a forças externas, por exemplo). Para corrigir sua distância devemos movê-las (projetar) de forma a satisfazer Eq. 2.1, como ilustrado na Figura 1.

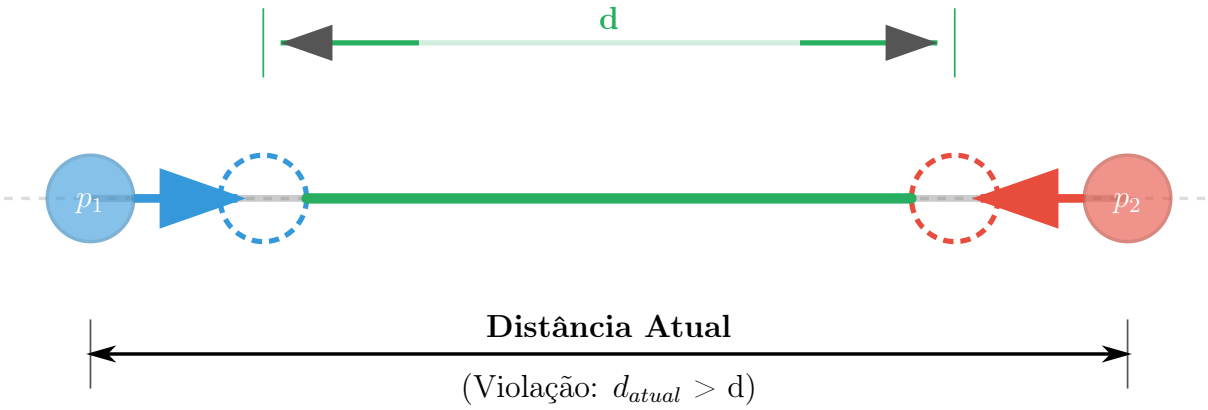


Figura 1 – A distância atual entre as partículas p_1 e p_2 está inválida e deve ser corrigida por projeção.

Jakobsen (2001) também descreve um esquema de articulação (dobradiça) fazendo que objetos compartilhem a mesma partícula como ilustrado na Figura 2. Dessa forma, seria possível modelar um corpo humanoide, Jakobsen (2001) também mostra que para realismo adicional pode-se limitar o ângulo de rotação de articulações através de restrições angulares satisfazendo o produto interno abaixo:

$$(x_2 - x_0) \cdot (x_1 - x_0) < \alpha.$$

Um método alternativo e simples que Jakobsen (2001) também mostra é usar restrições de distância auxiliares, que não foi usada neste trabalho, onde a distância entre as extremidades i e j não seja maior que um valor d_{min}:

$$|\vec{x}_i - \vec{x}_j| > d_{min}$$

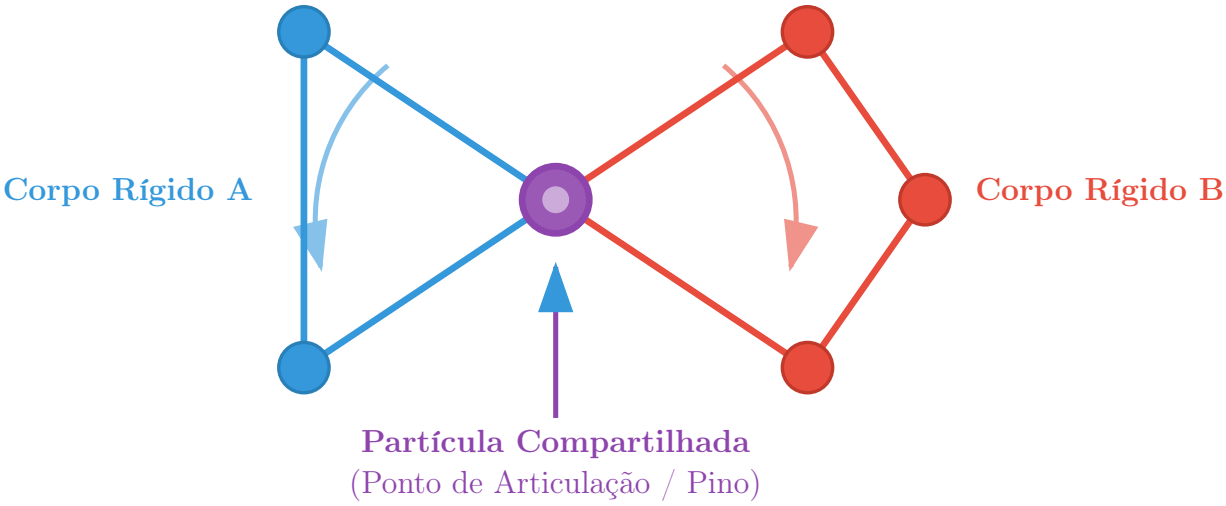


Figura 2 – A articulação é formada simplesmente fazendo os dois corpos compartilharem a mesma partícula

2.7 RESOLVENDO RESTRIÇÕES CONCORRENTES POR RELAXAMENTO

Na prática, uma simulação pode conter muitas restrições de todos os tipos vistos anteriormente. Para satisfazer todas elas devemos resolver todas elas sequencialmente, como as restrições entre partículas são interdependentes, não é possível satisfazê-las todas simultaneamente de maneira exata em um único passo.

Jakobsen (2001) propõe um método iterativo de relaxamento, também conhecido como relaxamento de *Gauss-Seidel*, para resolver as restrições de forma aproximada. É uma abordagem de solução indireta por iteração local que consiste em aplicar pequenas correções de posição para cada par de partículas conectado, repetindo o procedimento diversas vezes até que todas as restrições estejam aproximadamente satisfeitas. Cada iteração contribui para reduzir o erro acumulado, e a convergência ocorre rapidamente mesmo com poucas iterações (geralmente entre 3 e 5).

Apesar dessa abordagem parecer ingênua, ao resolver todas restrições localmente e repetir, o sistema global do sistema converge para uma configuração que satisfaz todas restrições. Quanto maior o número de iterações, mais rápido o sistema irá convergir para solução ideal e também a animação irá parecer mais rígida para o usuário.

Deve-se encontrar o movimento mínimo que satisfaça cada restrição, para restrição linear podemos fazer como no Algoritmo 1 que irá separar ou aproximar as partículas de

tal forma que satisfaçam a distância d. Onde ϵ representa o tamanho de passo local para

Algoritmo 1: Relaxamento para restrição linear

$\vec{\delta} \leftarrow \vec{x}_2 - \vec{x}_1$
$c \leftarrow \frac{\|\vec{\delta}\| - d}{\|\vec{\delta}\|}$
$\vec{x}_1 \leftarrow \vec{x}_1 - \epsilon\, \vec{\delta}\, c$
$\vec{x}_2 \leftarrow \vec{x}_2 + \epsilon\, \vec{\delta}\, c$

convergência para solução ideal. Quando $\epsilon = 1$ as partículas são movidas imediatamente para posição correta. Essa situação é comparável a um sistema de molas interconectadas entre partículas de rigidez que tendem para o infinito ou a uma haste rígida separando as duas partículas.

Jakobsen (2001) utiliza um modelo baseado em partículas e restrições geométricas, evitando explicitamente a solução de equações diferenciais rígidas e instáveis. Em vez disso, correções iterativas são aplicadas diretamente às posições das partículas, proporcionando estabilidade numérica elevada e comportamento visualmente plausível mesmo sob passos de tempo grandes.

2.8 POLÍGONOS, POLIEDROS E POLITOPOS

Um polígono é um objeto geométrico plano e fechado, formada por segmentos de reta (lados) que não se cruzam, unindo-se apenas nas extremidades (vértices). Os polígonos possuem pelo menos três lados e ângulos, não podendo ter superfícies curvas ou abertas. Exemplos comuns incluem triângulos, quadrados e hexágonos.

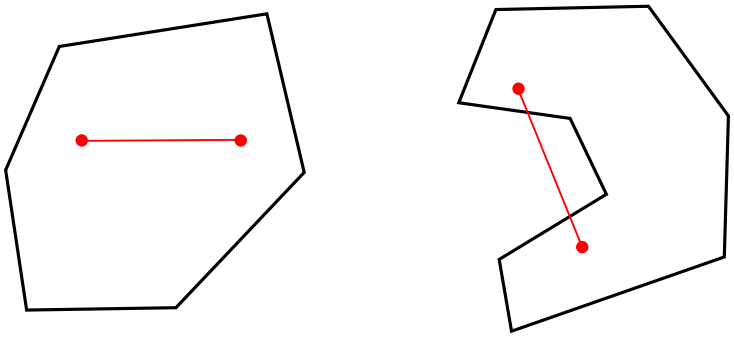


Figura 3 – Comparação entre polígono convexo (à esquerda) e polígono côncavo (à direita).

Um polígono é considerado convexo se, para quaisquer dois pontos contidos nessa forma, todo o segmento que os une também estiver contido nela, como ilustrado na Figura 3. Uma consequência prática é que, para qualquer linha que atravesse o objeto, esta o intersecta em no máximo dois pontos. Formas não convexas podem ser tratadas como

composições de múltiplas partes convexas, o que permite a aplicação direta de algoritmos especializados.

Um poliedro é um sólido geométrico tridimensional fechado, limitado por um número finito de faces planas que são polígonos. As interseções das faces são as arestas do poliedro. As interseções das arestas são os vértices do poliedro. Cada face é uma região poligonal contendo n lados. Exemplos comuns incluem cubos, pirâmides e prismas.

Poliedros convexos são aqueles cujos ângulos diedrais formados por planos adjacentes têm medidas menores do que 180 graus. Outra definição: Dados quaisquer dois pontos de um poliedro convexo, o segmento que tem esses pontos como extremidades, deverá estar inteiramente contido no poliedro.

Um politopo é um objeto geométrico com lados planos (faces). Os politopos são a generalização dos poliedros tridimensionais para qualquer número de dimensões. Os politopos podem existir em qualquer número geral de dimensão n. Por exemplo, um polígono é um 2-politopo e um poliedro é um 3-politopo.

3 DETECÇÃO DE COLISÕES

A detecção de colisões é um componente fundamental em sistemas de simulação física e em animações baseadas em partículas. Esse processo identifica quando dois ou mais objetos entram em contato, determina pontos de interseção e, quando necessário, fornece informações como vetores de penetração e normais que servirão de entrada para a etapa subsequente de resposta física. Em contextos interativos em tempo real, a precisão geométrica absoluta costuma ser sacrificada em favor da eficiência computacional, desde que o comportamento resultante se mantenha visualmente verossímil.

Este capítulo introduz os principais conceitos utilizados no âmbito deste trabalho, descrevendo as representações geométricas mais comuns para objetos convexos e apresentando dois algoritmos amplamente empregados em detecção de colisões: o Teorema do Eixo Separador (SAT, do inglês *Separating Axis Theorem*) e o algoritmo Gilbert-Johnson-Keerthi (GJK). Ambos operam eficientemente em formas convexas e constituem a base de vários motores físicos modernos.

3.1 TEOREMA DO EIXO SEPARADOR (SAT)

O Teorema do Eixo Separador (SAT, do inglês *Separating Axis Theorem*) é um dos métodos mais difundidos para detecção de colisão entre polígonos convexos. É um algoritmo genérico, rápido, que pode remover a necessidade de implementar código específico de detecção de colisão para cada par de tipos de forma, reduzindo assim o código e a manutenção. O teorema decorre de um resultado fundamental da análise convexa que afirma que:

Teorema 1. *Dois polígonos convexos A e B não se intersectam se, e somente se, existe um hiperplano separador P tal que A está de um lado de P e B está do outro.*

O teste do eixo separador decorre intuitivamente do Teorema 1, pois dois objetos convexos não podem "se curvar" um ao redor do outro. Assim, quando não estão se intersectando, haverá um espaço entre eles no qual um hiperplano pode ser inserido para separar os dois objetos. Quando os objetos estão se intersectando, nenhuma superfície pode ser inserida entre eles para separá-los. (ERICSON, 2004b)

Dado um hiperplano P que separa A e B, um eixo separador é uma reta L perpendicular a P. Esse eixo é denominado separador porque as projeções ortogonais de A e B sobre L resultam em dois intervalos que não se sobrepõem. Veja a Figura 4. Como esses dois intervalos não se sobrepõem, conclui-se que as geometrias são disjuntas. Como um eixo separador existe se, e somente se, existir um hiperplano separador, pode-se testar

qualquer um dos dois. No entanto, na prática, verifica-se que é melhor testar a separação em um eixo, pois isso resulta em um teste menos custoso.

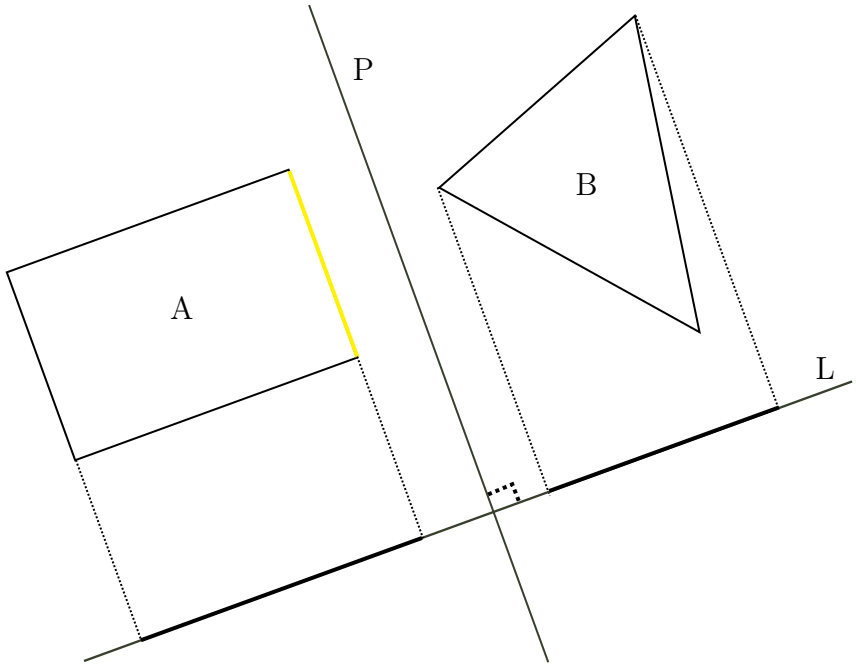


Figura 4 – Aresta escolhida em amarelo, eixo separador perpendicular à face. Fonte: Autor

Eixos separadores são fáceis de identificar por inspeção. No entanto, para uma implementação, é importante conseguir limitar automaticamente um número infinito de eixos potencialmente separadores a apenas alguns eixos a serem testados. Para poliedros convexos, é possível reduzir drasticamente o número de eixos a testar. Ignorando a ordem, dois objetos poliédricos podem entrar em contato de diferentes maneiras, como face-face, face-aresta, face-vértice, aresta-aresta, aresta-vértice ou vértice-vértice (ERICSON, 2004b). Como vértices podem ser considerados parte das arestas, no caso bidimensional, isso implica que os eixos candidatos podem ser obtidos a partir das normais das arestas dos polígonos.

Dado um eixo unitário $\hat{n}$ e um conjunto de vértices $\{v\}$ de um polígono, a projeção gera um intervalo escalar $[min, max]$ definido por:

$$min = \min_i(\hat{n} \cdot \vec{v}_i)$$
$$max = \max_i(\hat{n} \cdot \vec{v}_i).$$

A verificação de sobreposição entre dois intervalos $[min_A, max_A]$ e $[min_B, max_B]$ é dada pela condição:

$$\text{Sobreposição} \iff max_A \geq min_B \quad \wedge \quad max_B \geq min_A,$$

como ilustrado na Figura 4. Se essa condição falhar em qualquer eixo candidato, os objetos estão separados e o algoritmo pode encerrar imediatamente (dyn4j, 2010).

O método possui complexidade quadrática no número de arestas (ERICSON, 2004b), o que pode reduzir sua praticidade em relação a alternativas como o GJK, especialmente em dimensões mais altas.

3.2 ALGORITMO DE GILBERT-JOHNSON-KEERTHI (GJK)

O algoritmo de Gilbert-Johnson-Keerthi (GJK) é um algoritmo clássico para calcular distância euclidiana entre dois politopos convexos, um problema fundamental em áreas como computação gráfica, simulação física, robótica, planejamento de trajetórias e sistemas de detecção de colisão.

Originalmente o algoritmo permite calcular a distância mínima entre politopos convexos em $\mathbb{R}^m$, definidos pelo número de vértices em sua fronteira, com complexidade linear no número de vértices dos objetos. O algoritmo de GJK é amplamente utilizado em motores físicos devido à sua eficiência e robustez, especialmente em ambientes tridimensionais (GILBERT; JOHNSON; KEERTHI, 1988).

Neste trabalho, detalharemos uma versão aprimorada do GJK proposta por Gilbert e Foo (1990), que é capaz de tratar qualquer objeto convexo.

Distância Euclidiana entre Conjuntos Convexos

Sejam A e B dois conjuntos convexos compactos em $\mathbb{R}^m$. A distância euclidiana entre eles é definida como:

$$d(A, B) = \min_{a \in A, b \in B} \|a - b\|$$

Quando A e B se intersectam, $d(A, B) = 0$.

O Fecho Convexo

Seja $A \subset \mathbb{R}^m$ um conjunto não vazio de pontos. O fecho convexo de A, denotado por $\text{conv}(A)$ ou $\text{CH}(A)$, é definido como o menor conjunto convexo que contém A.

Geometricamente, o fecho convexo corresponde ao "invólucro" mínimo que envolve completamente o conjunto A. No caso bidimensional, pode ser interpretado como o polígono convexo de menor área que contém todos os pontos de A, como ilustrado pela Figura 5; em três dimensões, como o menor poliedro convexo que os contém.

Soma de Minkowski

O algoritmo de GJK fundamenta-se na soma de Minkowski entre dois conjuntos convexos A e B, definida como

$$A + B = \{\vec{x} + \vec{y} \mid \vec{x} \in A,\ \vec{y} \in B\},$$

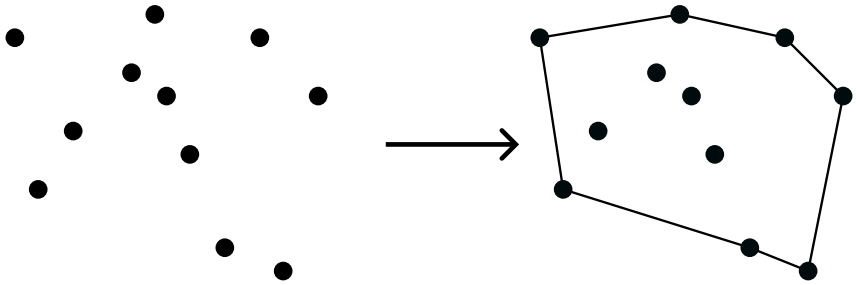


Figura 5 – Fecho convexo de um conjunto de pontos.

onde $\vec{x}$ e $\vec{y}$ representam *vetores posição* associados a pontos pertencentes aos conjuntos A e B, respectivamente. Observe que é uma prática comum, principalmente nos textos sobre detecção de colisão, representar um *ponto* P por meio do *vetor posição* $\vec{p} = P - O$, onde O é a origem do sistema de coordenadas.

Do ponto de vista geométrico, a soma de Minkowski pode ser interpretada como o conjunto de todos os pontos obtidos ao transladar o objeto B por cada vetor posição pertencente a A, mantendo sua forma e orientação, ou seja, faz-se uma cópia do objeto B centrado em cada ponto de A, como ilustrado na Figura 6.

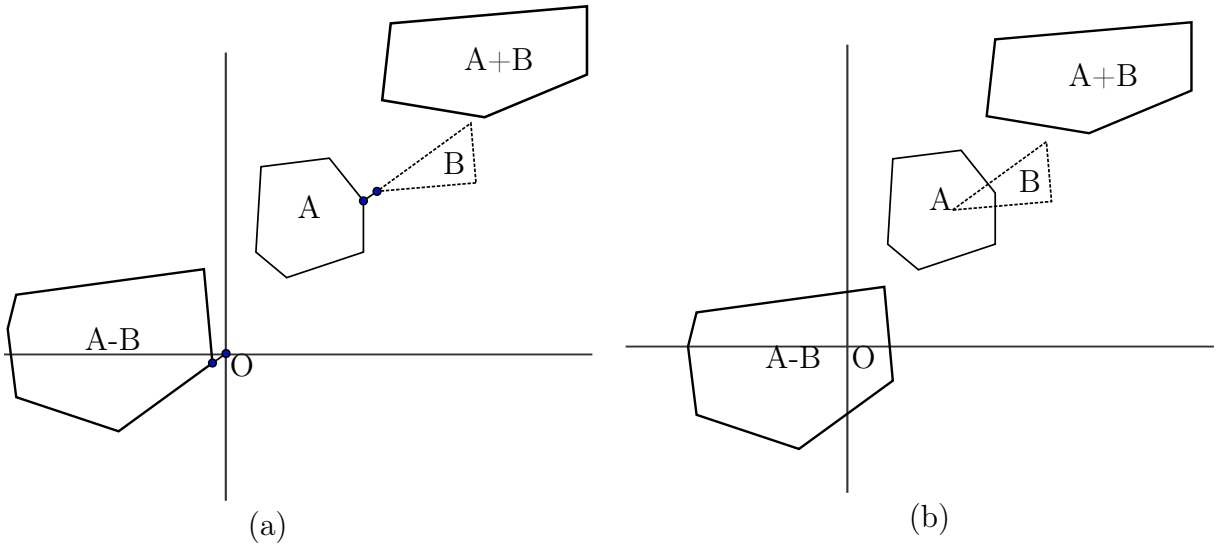


Figura 6 – Soma de Minkowski, entre os conjuntos A e B.

Uma propriedade muito útil da soma de Minkowski é o fato de que a soma de dois objetos convexos é um objeto convexo. Para um objeto A, usamos a notação $-A$ para denotar a reflexão de A sobre a origem O. A diferença de Minkowski $A - B$ pode ser obtida calculando a soma de Minkowski de A e $-B$

$$A - B = A + (-B) = \{\vec{x} - \vec{y} \mid \vec{x} \in A, \vec{y} \in B\}.$$

A diferença de Minkowski também é chamada de configuração do espaço de obstáculos (CSO, do inglês *Configuration Space Obstacles*). Neste espaço a distância entre os objetos

A e B é igual à distância entre a origem O e o CSO, como ilustrado na Figura 6 (a). Logo, A e B se intersectam se, e somente se, sua CSO contém a origem (LINAHAN, 2015), ou seja:

$$d(A, B) = \min_{c \in CSO} \|c\|.$$

Isso se deve ao fato de que se existe um ponto $p_A \subset A$ igual a $p_B \subset B$, então $p_A - p_B = \vec{0}$ é testemunha de que há pelo menos um ponto do CSO que coincide com a origem O. Compare a Figura 6 (a) e (b). Portanto, o problema de proximidade entre dois objetos reduz-se ao problema de encontrar o ponto de menor norma da diferença de Minkowski.

Função suporte e ponto de suporte

Um ponto interessante do algoritmo é que não é necessário calcular explicitamente o CSO, apenas uma amostra do CSO, sob demanda, usando um mapeamento de suporte. Uma função de suporte é definida tal que: seja A um conjunto qualquer, o suporte de A na direção de $\vec{d}$ retorna o ponto que pertence a superfície de A mais distante da origem na direção $\vec{d}$, esse ponto é chamado de ponto de suporte. Gilbert e Foo (1990) mostraram que é possível representar qualquer objeto convexo com a função suporte correta. Por exemplo, para um um politopo A, podemos escrever a seguinte função de suporte

$$S_A(\vec{d}) = \arg\max_{\vec{a} \in A} (\vec{d} \cdot \vec{a}). \tag{3.1}$$

Uma propriedade fundamental é que funções de suporte permitem realizar operações entre conjuntos convexos

$$S_{A+B}(\vec{d}) = S_A(\vec{d}) + S_B(\vec{d}),$$
$$S_{-A}(\vec{d}) = -S_A(-\vec{d}).$$

Dessa forma, seja D a diferença de Minkowski de dois objetos convexos A e B, o suporte de D na direção $\vec{d}$ é a diferença dos suportes de A e B: $S_{A-B}(\vec{d}) = S_A(\vec{d}) - S_B(-\vec{d})$, como ilustrado pela Figura 7.

A busca pelo ponto mais próximo da origem dentro do CSO é sustentado pelo teorema de Carathéodory (ROCKAFELLAR, 1970). O teorema afirma que se um ponto $\vec{x}$ está no interior do fecho convexo de um conjunto P em $\mathbb{R}^m$ então $\vec{x}$ está no interior de um simplex com vértices em P.

Um simplex (plural: *simplices*) é definido como um politopo de $k + 1$ vértices em um espaço de dimensão k. Dessa forma um ponto é 0-simplex, um segmento de reta é 1-simplex, um triângulo é 2-simplex, um tetraedro é 3-simplex, como na Figura 8.

O teorema de Carathéodory em 2 dimensões afirma que podemos construir um triângulo usando pontos em P que envolve qualquer ponto no interior do fecho convexo de P.

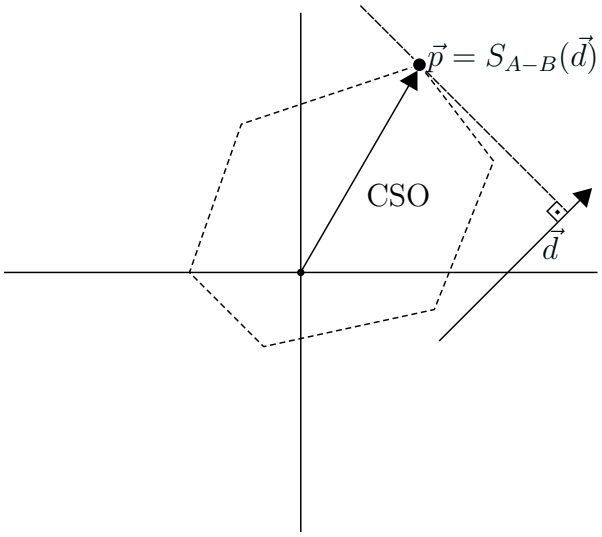


Figura 7 – A função suporte na forma implícita $A - B$ ao longo da direção $\vec{d}$.

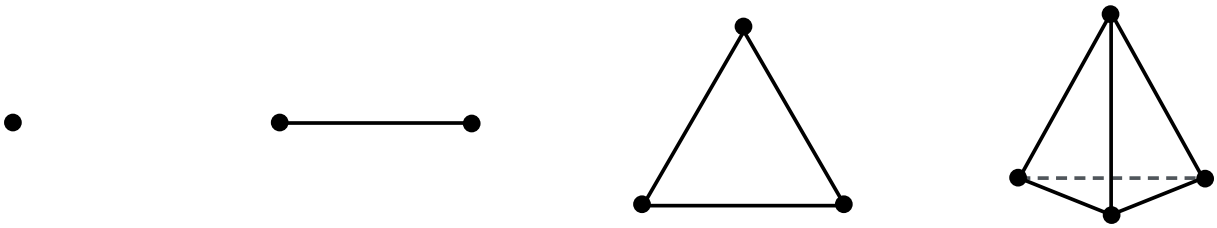


Figura 8 – Da esquerda para direita 0-simplex, 1-simplex, 2-simplex e 3-simplex

Por exemplo, seja $P = (0,0),(0,1),(1,0),(1,1)$. O fecho convexo deste conjunto é um quadrado. Seja $\vec{x} = (1/4, 1/4)$ no interior do fecho convexo de P. Podemos então construir um triângulo $(0,0),(0,1),(1,0) = P'$ que envolve $\vec{x}$, como ilustrado na Figura 9. (Wikipedia contributors, 2025)

O algoritmo

O algoritmo GJK constrói iterativamente simplices $Q_1, Q_2, Q_3, ..., Q_n$ dentro do CSO, até chegar a um determinado simplex $Q_j = Q_{j+1}$, determinando que não houve colisão. Alternativamente o ponto $\vec{p}$ mais próximo da origem, pertencente a este simplex, constitui uma aproximação sucessivamente melhor da distância mínima entre os objetos. Caso esta seja suficientemente próximo de 0, então houve colisão.

O algoritmo tenta encontrar o simplex mais próximo da origem procurando um ponto de suporte $\vec{s}$ usando a direção $\vec{d}$, que aponta de P para a origem. Dessa forma:

- Se $\vec{s} \cdot \vec{d} > 0$, então existe um ponto do CSO na direção da origem.
- Se $\vec{s} \cdot \vec{d} = 0$, então o ponto de suporte é ortogonal à direção de busca, indicando que a origem está tangente ao CSO.

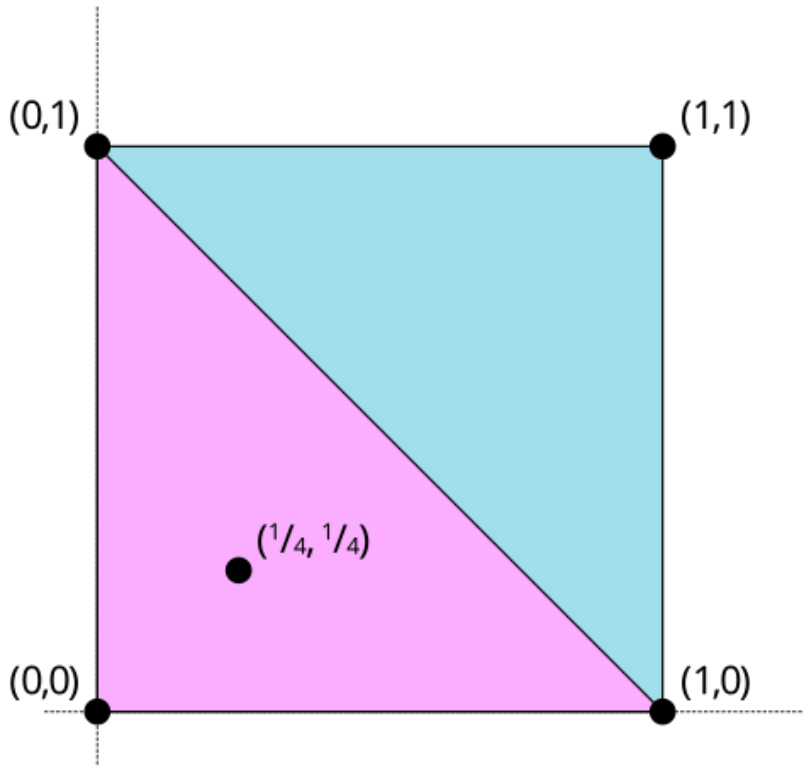


Figura 9 – Ilustração do teorema de Carathéodory. O ponto $\vec{x}$ está no interior do fecho convexo de P, e também está no interior do fecho convexo de P'.

- Se $\vec{s} \cdot \vec{d} < 0$, então o ponto mais extremo do CSO na direção $\vec{d}$ não ultrapassa a origem. Consequentemente, nenhum ponto do CSO pode estar além da origem nessa direção, implicando que a origem não pertence ao CSO e que os objetos A e B não se intersectam.

Algoritmo 2: GJK

Input: Polígonos convexos A e B, direção inicial $\vec{d}$
Output: Distância euclidiana entre A e B
$\vec{s} \leftarrow S_A(\vec{d}) - S_B(-\vec{d})$
$Q \leftarrow \{\vec{s}\}$
$\vec{p} \leftarrow \vec{s}$
$\vec{d} \leftarrow -\vec{s}$
while $\|\vec{p}\| > \epsilon$ **do**
  $\vec{s} \leftarrow S_A(\vec{d}) - S_B(-\vec{d})$
  if $\vec{s} \cdot \vec{d} \leq \epsilon$ **then**
    `// A e B não se intersectam`
    return $\|\vec{p}\|$
  $Q, \vec{d}, \vec{p} \leftarrow \text{NearestSimplex}(Q \cup \{\vec{s}\})$
return $\|\vec{p}\|$ `// A e B se intersectam`

O Algoritmo 2 inicia com a construção de um simplex Q contendo um ponto de suporte $\vec{s}$ do CSO em uma direção inicial arbitrária (comumente escolhida como o vetor que liga o centro geométrico de A ao centro geométrico de B). Em seguida, define-se a nova direção $\vec{d}$ como o vetor oposto a $\vec{s}$, orientando a busca em direção à origem.

O laço principal consiste em três etapas fundamentais. Primeiramente, calcula-se um novo ponto de suporte na direção atual $\vec{d}$. Caso adicionar o novo ponto de suporte não melhore a aproximação para origem, conclui-se que não há avanço em direção à origem. Introduz-se um termo de tolerância ϵ a fim de evitar laços infinitos e limitar o erro entre a distância de separação calculada e a distância real. Como a cada laço o ponto de suporte consulta um novo vértice, e há um número limitado de vértices a serem consumidos, então o algoritmo garantidamente termina.

Caso contrário, o ponto $\vec{s}$ é adicionado ao simplex Q. Em seguida, aplica-se o procedimento NearestSimplex(Q), que determina o subconjunto convexo mínimo de Q cujo fecho convexo contém o ponto mais próximo da origem, além de atualizar a direção de busca $\vec{d}$ para apontar de $\vec{p}$ à origem e de atualizar o ponto mais próximo $\vec{p}$. A complexidade assintótica de *NearestSimplex* é constante e baixa, igual a cardinalidade de $Q \cup \{\vec{s}\}$, que é no máximo 4 em 3D e 3 em 2D.

Subalgoritmo de distância de Ericson

No Algoritmo 2, usa-se a função NearestSimplex para calcular o simplex mais próximo da origem dado um conjunto de pontos. Na descrição original, esta função era implementada pelo subalgoritmo de Johnson, que resolve um sistema de equações lineares para combinação de vértices, faces e arestas. Com o passar dos anos, esse subalgoritmo foi substituído por outros mais sofisticados como, por exemplo, o subalgoritmo de Ericson apresentado na SIGGRAPH 2004 (ERICSON, 2004a), e que se baseia numa abordagem geométrica ao invés de algébrica, com otimização usando regiões.

A função NearestSimplex implementada conforme o subalgoritmo de Ericson recebe como argumento um conjunto de pontos que pode ter cardinalidade 1, 2 ou 3 quando aplicado a problemas em 2D. Ela retorna um simplex representado por um conjunto Q com 1, 2 ou 3 pontos, um novo vetor de direção $\vec{d}$ e o ponto $\vec{p}$ do simplex mais próximo da origem.

Em particular, veremos que, sempre que NearestSimplex retorna um conjunto com 3 pontos, o algoritmo GJK termina, já que este triângulo ou contém a origem, ou garantidamente não há interseção entre A e B.

Para cada dimensão do simplex Q é feito uma análise. Neste trabalho iremos trabalhar apenas com objetos em 2D, por isso trataremos apenas dos casos 1-simplex e 2-simplex. Para descrever o subalgoritmo de Ericson, denotamos a origem por O e os vértices do simplex por R, S e T, em ordem alfabética, sendo R o ponto mais recente. Associamos a

cada ponto seu vetor posição em relação à origem:

$$\vec{r} = \vec{OR}, \qquad \vec{s} = \vec{OS}, \qquad \vec{t} = \vec{OT}.$$

O caso 0-simplex é impossível, pois, como visto no Algoritmo 2, dentro do laço principal do GJK, o simplex é sempre atualizado com um novo ponto de suporte, e o simplex inicial já contém um ponto de suporte. Portanto, o simplex mínimo possível é um 1-simplex.

No caso 1-simplex temos um simplex formado pelos pontos R e S que dividem o espaço em três regiões, como ilustrado pela Figura 10. A região do vértice R corresponde a todos os pontos P tais que $(P - R) \cdot (S - R) < 0$, e similarmente para a região do vértice S. Consequentemente, a região da aresta $\vec{rs} = \vec{s} - \vec{r}$ corresponde a todos os pontos que estão no corredor entre R e S.

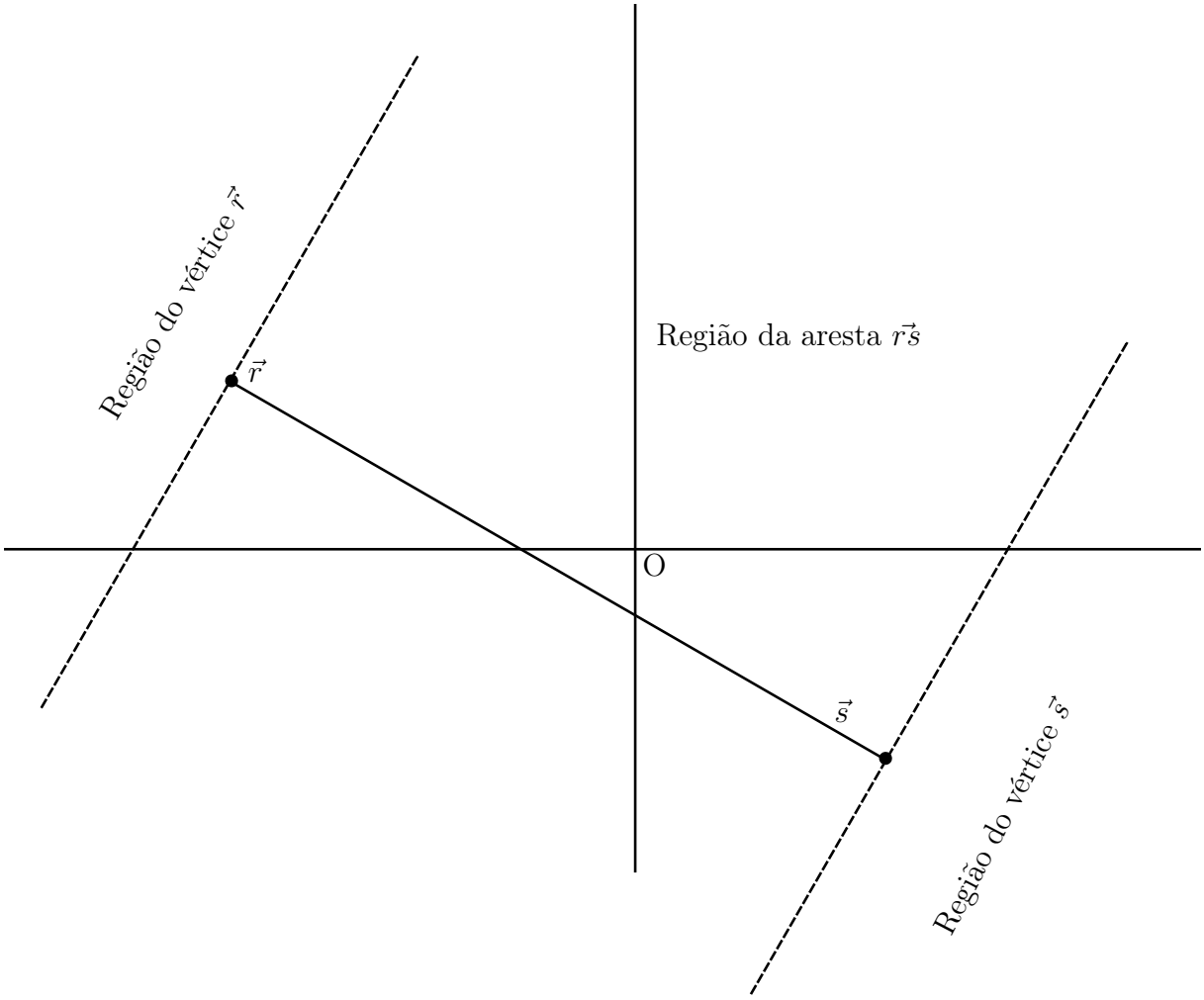


Figura 10 – Divisão de regiões para o caso 1-simplex.

Se a origem está na região de R, então S é descartado e a nova direção de busca $\vec{d}$ é o vetor $\vec{ro}$. Similarmente, para o caso onde origem está na região de S, a nova direção $\vec{d}$ é o vetor $\vec{so}$. Caso a origem se encontre na região da aresta $\vec{rs}$, a nova direção de busca deve ser perpendicular ao vetor $\vec{rs}$ e orientada em direção à origem. Essa direção pode

ser obtida utilizando o produto triplo vetorial

$$\vec{d} = (\vec{rs} \times \vec{ro}) \times \vec{rs}.$$

O produto triplo gera um vetor perpendicular a $\vec{rs}$ e contido no plano definido por $\vec{rs}$ e $\vec{ro}$, e orientado em direção à origem. Entretanto, como as regiões de vértice estão no limite do CSO, se a origem estiver contida nas regiões de vértice então a origem está além do limite do CSO. Portanto, não há interseção entre os objetos A e B.

Caso a iteração termine nesse passo, o ponto mais próximo da origem é obtido projetando a origem sobre o segmento. O parâmetro de projeção da origem sobre o segmento é dado por:

$$t = \frac{(-\vec{r}) \cdot (\vec{rs})}{\|\vec{rs}\|^2}.$$

e o ponto mais próximo é

$$\vec{p} = \begin{cases} \vec{r}, & t \leq 0, \\ \vec{s}, & t \geq 1, \\ \vec{r} + t(\vec{rs}), & 0 < t < 1. \end{cases}$$

O Algoritmo 3 sumariza a análise do caso 1-simplex.

Algoritmo 3: Caso 1-simplex

Input: Simplex $Q = \{\vec{r}, \vec{s}\}$
Output: Simplex Q, direção de busca $\vec{d}$ e ponto mais próximo $\vec{p}$
$\vec{rs} \leftarrow \vec{s} - \vec{r}$
$\vec{ro} \leftarrow -\vec{r}$
$\vec{sr} \leftarrow \vec{r} - \vec{s}$
$\vec{so} \leftarrow -\vec{s}$
if $\vec{rs} \cdot \vec{ro} \leq 0$ **then**
  return $\{\vec{r}\}$, $\vec{ro}$, $\vec{r}$ `// Origem na região do vértice R`
else if $\vec{sr} \cdot \vec{so} \leq 0$ **then**
  return $\{\vec{s}\}$, $\vec{so}$, $\vec{s}$ `// Origem na região do vértice S`
$\vec{d} \leftarrow (\vec{rs} \times \vec{ro}) \times \vec{rs}$
$\vec{p} \leftarrow$ Calcular ponto mais próximo da aresta rs
return Q, $\vec{d}$, $\vec{p}$ `// Origem na região da aresta`

Para o caso 2-simplex, temos um triângulo que divide o espaço em 7 regiões: regiões dos vértices R, S, T, regiões das arestas $\vec{rs}, \vec{st}, \vec{rt}$, e o interior do $\triangle RST$, como ilustrado na Figura 11. Como estamos trabalhando em 2D, se a origem está contida no triângulo, então podemos afirmar que os dois objetos se intersectam. Caso contrário, dependendo de em qual região ele cai, manteremos o vértice ou aresta correspondente e iteraremos novamente, como no Algoritmo 4. Em 3D seria necessário considerar a posição relativa da origem em relação ao plano do triângulo (para cima ou para baixo).

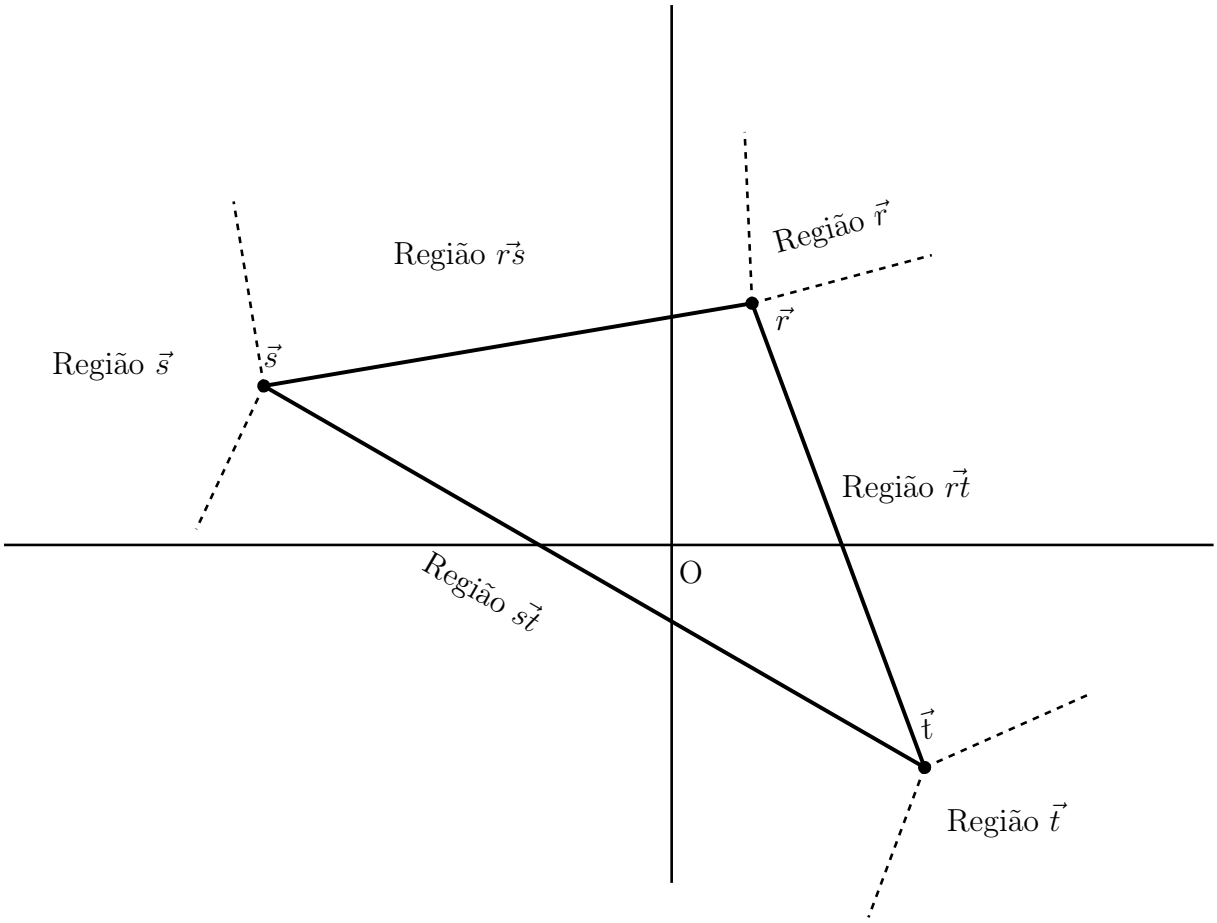


Figura 11 – Regiões para o caso 2-simplex.

No entanto, o processo de encontrar o simplex ideal elimina a necessidade de verificar quatro dessas regiões. Como o ponto de suporte T foi o primeiro a ser adicionado, S foi o segundo e R foi o terceiro, então o ponto de suporte R foi adicionado como resultado de encontrar o vetor normal à aresta $\vec{st}$. Isso significa que a região da aresta $\vec{st}$ não pode conter a origem. As três regiões do vértice também não podem conter a origem, uma vez que essas regiões estão além dos limites do CSO, que foram derivadas dos pontos de suporte. Isso deixa apenas as regiões $\vec{rs}$, $\vec{rt}$ e o interior do $\triangle RST$ para investigar.

Dessa forma, para determinar em qual região do simplex a origem se encontra, analisam-se inicialmente as regiões associadas às arestas incidentes em R. O vetor normal ao plano do triângulo é obtido por

$$\vec{n} = \vec{rs} \times \vec{rt}.$$

A direção perpendicular à aresta $\vec{rs}$ no plano do triângulo é dada por

$$\vec{d_1} = \vec{n} \times \vec{rs}.$$

Se

$$\vec{d_1} \cdot \vec{ro} > 0,$$

então a origem encontra-se na região associada à aresta $\vec{rs}$. Nesse caso, o simplex é reduzido ao segmento $\{R, S\}$ e o algoritmo passa a executar o caso 1-simplex.

De maneira análoga, define-se

$$\vec{d_2} = \vec{rt} \times \vec{n}.$$

Se

$$\vec{d_2} \cdot \vec{ro} > 0,$$

então a origem encontra-se na região associada à aresta $\vec{rt}$, e o simplex é reduzido ao segmento $\{R, T\}$ e o algoritmo passa a executar o caso 1-simplex. Se ambos os casos anteriores forem falsos, a origem encontra-se no interior do triângulo.

Algoritmo 4: Caso 2-simplex

Input: Simplex $Q = \{\vec{r}, \vec{s}, \vec{t}\}$
Output: Simplex Q, direção de busca $\vec{d}$ e ponto mais próximo $\vec{p}$
$\vec{rs} \leftarrow \vec{s} - \vec{r}$
$\vec{rt} \leftarrow \vec{t} - \vec{r}$
$\vec{ro} \leftarrow -\vec{r}$
$\vec{n} \leftarrow \vec{rs} \times \vec{rt}$
$\vec{d_1} \leftarrow \vec{n} \times \vec{rs}$
if $\vec{d_1} \cdot \vec{ro} > 0$ **then**
  `// Origem na região da aresta` $\vec{rs}$
  return executar Caso 1-Simplex($\{\vec{r}, \vec{s}\}$)
$\vec{d_2} \leftarrow \vec{rt} \times \vec{n}$
if $\vec{d_2} \cdot \vec{ro} > 0$ **then**
  `// Origem na região da aresta` $\vec{rt}$
  return executar Caso 1-Simplex($\{\vec{r}, \vec{t}\}$)
`// Há interseção, distância 0`
return Q, $\vec{d}$, 0

Boolean GJK

Dependendo do problema a ser resolvido, é possível modificar o GJK para resolver um problema de decisão e determinar se há ou não interseção entre dois objetos A e B, conhecido como Boolean GJK (BGJK). Este algoritmo se baseia no fato de que A e B se intersectam se, e somente se, sua CSO contém a origem. Desta forma o objetivo do GJK se reduz a determinar se a CSO contém ou não a origem.

Neste trabalho foi implementada uma versão do BGJK apresentada por Muratori (2006), como o Algoritmo 7, que é baseado nos avanços de Gilbert e Foo (1990) e Ericson (2004a). Na formulação de Muratori, o simplex é manipulado com o objetivo exclusivo de verificar se ele contém a origem, testando regiões da mesma forma que Ericson (2004a). Seu critério de parada é mais simples: (1) não há necessidade de calcular explicitamente

o ponto mais próximo da origem; (2) o caso 1-simplex sempre retorna Falso, pois não é possível garantir que a origem esteja contida no CSO. Veja o Algoritmo 5 e 6. O algoritmo termina quando a origem é encontrada dentro do simplex ou quando o simplex $Q_j = Q_{j+1}$, indicando que a origem não pertence ao CSO, como no Algoritmo 7.

Algoritmo 5: Caso 1-simplex BGJK

Input: Simplex $Q = \{\vec{r}, \vec{s}\}$
Output: Simplex Q, $\vec{d}$, False
$\vec{rs} \leftarrow \vec{s} - \vec{r}$
$\vec{ro} \leftarrow -\vec{r}$
$\vec{sr} \leftarrow \vec{r} - \vec{s}$
$\vec{so} \leftarrow -\vec{s}$
if $\vec{rs} \cdot \vec{ro} \leq 0$ **then**
  return $\{\vec{r}\}$, False, $\vec{ro}$
else if $\vec{sr} \cdot \vec{so} \leq 0$ **then**
  return $\{\vec{s}\}$, False, $\vec{so}$
$\vec{d} \leftarrow (\vec{rs} \times \vec{ro}) \times \vec{rs}$
return Q, $\vec{d}$, False

Algoritmo 6: Caso 2-simplex BGJK

Input: Simplex $Q = \{\vec{r}, \vec{s}, \vec{t}\}$
Output: Simplex Q, $\vec{d}$, True se origem está contida, False caso contrário
$\vec{rs} \leftarrow \vec{s} - \vec{r}$
$\vec{rt} \leftarrow \vec{t} - \vec{r}$
$\vec{ro} \leftarrow -\vec{r}$
$\vec{n} \leftarrow \vec{rs} \times \vec{rt}$
$\vec{d_1} \leftarrow \vec{n} \times \vec{rs}$
if $\vec{d_1} \cdot \vec{ro} > 0$ **then**
  `// Próximo laço irá executar caso 1-simplex`
  return $\{\vec{r}, \vec{s}\}$, False, $\vec{d_1}$
$\vec{d_2} \leftarrow \vec{rt} \times \vec{n}$
if $\vec{d_2} \cdot \vec{ro} > 0$ **then**
  `// Próximo laço irá executar caso-1-simplex`
  return $\{\vec{r}, \vec{t}\}$, False, $\vec{d_2}$
return Q, $\vec{d}$, True

Algoritmo 7: BGJK

Input: Polígonos convexos A e B, direção inicial $\vec{d}$
Output: Verdadeiro se A e B se intersectam, falso caso contrário
$\vec{s} \leftarrow S_A(\vec{d}) - S_B(-\vec{d})$
$Q \leftarrow \{\vec{s}\}$
$\vec{d} \leftarrow -\vec{s}$
while *True* **do**
  $\vec{s} \leftarrow S_A(\vec{d}) - S_B(-\vec{d})$
  if $\vec{s} \cdot \vec{d} \leq 0$ **then**
    `// A e B não se intersectam`
    return False
  Q, $\vec{d}$, *interseccao* $\leftarrow$ NearestSimplexBGJK($Q \cup \{\vec{s}\}$)
  if *interseccao* **then**
    `// A e B se intersectam`
    return True

É possível modificar esse algoritmo para usar variáveis de controle adicionais e parar com antecedência caso a origem esteja contida em regiões de vértice. Neste trabalho preferiu-se manter a formulação mais simples e se beneficiar da estrutura iterativa.

Existem certas configurações de objetos que podem levar a *loops* infinitos devido a erros de operação com ponto flutuante. Para evitar isso, pode-se empregar práticas de correções numéricas de erros no algoritmo. Muratori (2006) apresenta uma solução simples: introduz um critério de parada adicional, limitando o número máximo de iterações. Se esse limite for atingido, significa que ocorreram aproximações sucessivas do simplex e o algoritmo assume que os objetos estão em colisão. Embora essa abordagem possa resultar em falsos positivos em casos extremos, ela é eficaz para garantir a robustez do algoritmo em cenários práticos, onde tais configurações são raras.

4 RESPOSTA A COLISÕES

A resposta a colisões é uma etapa fundamental em qualquer sistema de simulação física interativa. Após detectar a interpenetração entre dois corpos, torna-se necessário aplicar um conjunto de correções que restaurem a plausibilidade física do movimento, evitando instabilidades numéricas.

Este capítulo discute os princípios clássicos e as formulações modernas de resposta a colisão, estabelecendo a relação entre os métodos geométricos de detecção e a abordagem baseada em partículas e restrições proposta por Jakobsen (2001), que fundamenta este trabalho.

4.1 MÉTODOS DINÂMICOS NA SIMULAÇÃO FÍSICA

A resposta a colisões consiste, essencialmente, em duas operações principais:

1. Correção de Posição: eliminar a interpenetração entre dois corpos.
2. Correção de Velocidade: remover ou ajustar componentes da velocidade que induziriam novo contato imediato.

A literatura apresenta diversas abordagens para modelar o movimento de corpos rígidos e deformáveis. Embora este trabalho se baseie no método simplificado de Jakobsen (2001), é importante contextualizar outras categorias amplamente utilizadas em motores físicos modernos:

Método do Impulso

O Método do Impulso trata colisões aplicando impulsos instantâneos que alteram diretamente as velocidades dos corpos para preservar o momento linear e angular. Essa abordagem é utilizada em motores como Havok (Havok, 2024) e Bullet (COUMANS, 2015). O impulso é calculado em função da velocidade relativa no ponto de contato, resultando em um método eficiente para simulações em tempo real.

Método de Penalidades

Nos Métodos de Penalidades, colisões são tratadas como interpenetrações que geram forças de repulsão proporcionais à profundidade de penetração alterando diretamente a aceleração. Essas forças geralmente seguem modelos de mola e amortecimento. Trata-se de um método simples, porém sensível à escolha dos parâmetros de rigidez, podendo causar instabilidade numérica.

Método Baseado em Restrições com Multiplicadores de Lagrange

Os métodos baseados em restrições formulam os contatos como equações que devem ser satisfeitas exatamente. São resolvidos usando multiplicadores de Lagrange, sendo uma abordagem robusta e adequada para sistemas complexos, mas que exige a solução de sistemas lineares, tornando sua aplicação onerosa em plataformas Web.

4.2 PROCESSO DE SEPARAÇÃO

A metodologia de Jakobsen (2001) pode ser compreendida como uma precursora da moderna dinâmica baseada em posição (PBD, do inglês *Position Based Dynamics*) (MÜLLER et al., 2007). Diferentemente das abordagens que atuam sobre a velocidade ou aceleração, Jakobsen (2001) trata colisões como restrições geométricas adicionais que devem ser satisfeitas durante o processo iterativo de relaxamento da simulação. Assim, objetos penetrando um ao outro são corrigidos exclusivamente por modificações de posição.

A resposta à colisão envolve dois passos principais. O primeiro consiste em separar os elementos geométricos (vértices, arestas ou faces) que se encontram em interpenetração, o que caracteriza um processo estritamente geométrico. O segundo passo corresponde a um processo iterativo de relaxamento, no qual os elementos afetados ajustam suas posições de acordo com as restrições impostas pelo sistema físico.

Para dois objetos convexos A e B em colisão, é necessário uma descrição da área de contato. É comum o esquema de detecção de colisão retornar os pontos de contato de cada objeto e o vetor de translação mínima (MTV, do inglês *Minimum Translation Vector*). O MTV $\vec{d}$ de um par de objetos que se intersectam é o vetor mais curto sobre o qual um dos objetos precisa ser transladado para separar o par.

Com essas informações devemos tratar duas configurações possíveis: colisão vértice-vértice ou colisão vértice-aresta.

Colisão Vértice-Vértice

Se o contato ocorre pontualmente entre duas partículas p e q, a correção é dividida igualmente (assumindo massas iguais):

$$\Delta\vec{p} = -\frac{1}{2}\vec{d}, \qquad \Delta\vec{q} = +\frac{1}{2}\vec{d}.$$

Colisão Vértice-Aresta

Esta é a configuração mais comum. Um vértice p de um corpo colide contra uma aresta definida pelas partículas $\vec{x}_1$ e $\vec{x}_2$ de outro corpo. O ponto de impacto p cai entre dois vértices $\vec{x}_1$ e $\vec{x}_2$ e o nosso objetivo é corrigir as suas posições para uma configuração válida $\vec{x_1'}$, $\vec{x_2'}$ de tal forma que garanta uma coerência geométrica, como mostra a Figura 12.

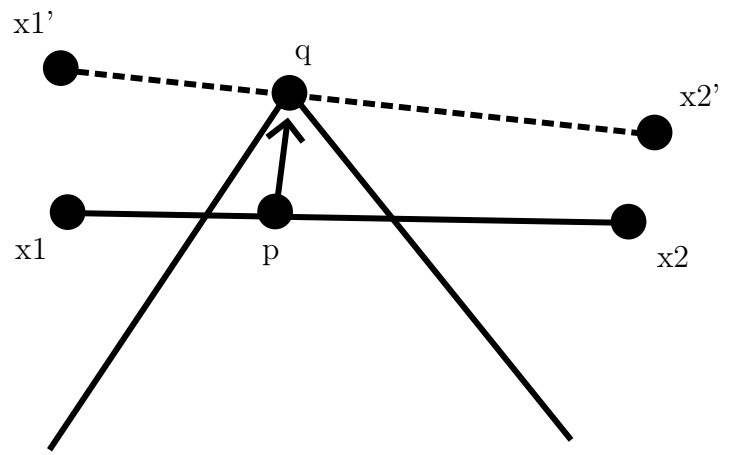


Figura 12 – Processo de separação caso colisão vértice-aresta. Correção das posição para configuração válida. Perceba que, como p está mais próximo de x_1, esse vértice é movido mais.

Logo, pela equação da reta, p pode ser descrito como uma interpolação linear

$$p = \alpha\vec{x}_1 + (1-\alpha)\vec{x}_2, \qquad 0 \leq \alpha \leq 1, \tag{4.1}$$

ou seja, quando $\alpha = 0, p = x_2$, quando $\alpha = 1, p = x_1$.

Durante a separação, a correção $\Delta\vec{p}$ deve alterar indiretamente $\vec{x}_1$ e $\vec{x}_2$ de forma proporcional a essa parametrização. A partir da Eq. 4.1, derivamos o valor de α calculando o local onde o ponto de contato p se encontra ao longo da aresta $\{x_1x_2\}$. Dessa forma, Jakobsen (2001) computa as novas posições movendo as partículas proporcionalmente a α:

$$\begin{aligned}
\vec{x}_1 &+= \alpha\Delta\vec{p} \\
\vec{x}_2 &+= (1-\alpha)\Delta\vec{p} \\
\alpha &= \frac{(\vec{p}-\vec{x}_2)\cdot(\vec{x}_1-\vec{x}_2)}{\|\vec{x}_1-\vec{x}_2\|^2}.
\end{aligned}$$

Isso garante que a geometria original é preservada e que o ponto $\vec{p}$, definido implicitamente pelos vértices da aresta, é deslocado exatamente pela quantidade desejada.

4.3 SAT COM CÁLCULO DO MTV

O SAT pode ser estendido para calcular o vetor de translação mínima. Em vez de encerrar antecipadamente quando uma sobreposição é detectada em um eixo, todos os eixos são testados quanto à sobreposição. Após todos os eixos terem sido testados, o eixo com a menor sobreposição (normalizada) pode ser usado como a normal de contato, e a magnitude da sobreposição pode ser utilizada para estimar a profundidade de penetração (ERICSON, 2004b). Como mostrado no Algoritmo 8.

,

Algoritmo 8: SAT com cálculo do MTV

Input: Polígonos convexos A e B
Output: $\vec{d}$ e δ, ou falso
$\vec{d} \leftarrow \vec{0}$
$\delta \leftarrow \infty$
foreach *aresta* $\hat{n}$ *de* A *e* B **do**
  $\hat{n} \leftarrow$ normal unitária de $\hat{n}$
  $p_1 \leftarrow$ Projeção de A
  $p_2 \leftarrow$ Projeção de B
  $\Delta \leftarrow$ sobreposição entre p_1 e p_2
  if $\Delta \leq \epsilon$ **then**
    return *False*
  if $\Delta < \delta$ **then**
    $\delta \leftarrow \Delta$
    $\vec{d} \leftarrow \hat{n}$
return $\vec{d}$, δ

4.4 ALGORITMO DE EXPANSÃO DE POLITOPOS (EPA)

O algoritmo de Expansão de Politopos (EPA, do inglês *Expanding Polytope Algorithm*) surge no contexto da evolução dos métodos de detecção de colisão entre objetos convexos, particularmente como uma extensão do algoritmo de GJK. No entanto, sua formulação não fornece diretamente informações necessárias para a resposta à colisão, como a profundidade de penetração ou a direção de separação. O EPA resolve o problema dual do GJK de encontrar a menor translação necessária para separar os objetos, o MTV. O desenvolvimento do EPA é geralmente atribuído a Bergen (2001), cujos trabalhos estabeleceram bases importantes para a manipulação geométrica do espaço de configuração (CSO).

Vimos no GJK que a distância de um par de objetos é um ponto na fronteira do CSO mais próximo da origem. A estratégia para encontrar o MTV é começar com um politopo que contém a origem e tem vértices que se encontram na fronteira do CSO, e expandir esse politopo adicionando vértices que também estão na fronteira. Para isso o GJK pode ser modificado para retornar o simplex final Q e usá-lo como entrada para o EPA.

O EPA é um algoritmo iterativo em que, para cada etapa da iteração, selecionamos a face do politopo mais próxima da origem. Sejam p o ponto mais próximo da origem que pertence à face selecionada, e $\vec{p}$ o seu vetor posição. Então, o comprimento de $\vec{p}$ é um limite inferior para a magnitude da profundidade de penetração, e a normal da face mais próxima da origem é uma aproximação para a direção de separação. Então a face selecionada é subdividida no ponto p e o extende até o ponto de suporte $\vec{w} = S_{CSO}(\vec{p})$. Observe que $||\vec{w}||$ é um limite *superior* para a profundidade de penetração. O processo continua até que o erro entre $\vec{p}$ e $\vec{w}$ seja menor que um valor de tolerância ϵ ou até que tenham sido feitas aproximações sucessivas suficientes. Veja a Figura 13 para uma representação visual do processo de expansão de politopos.

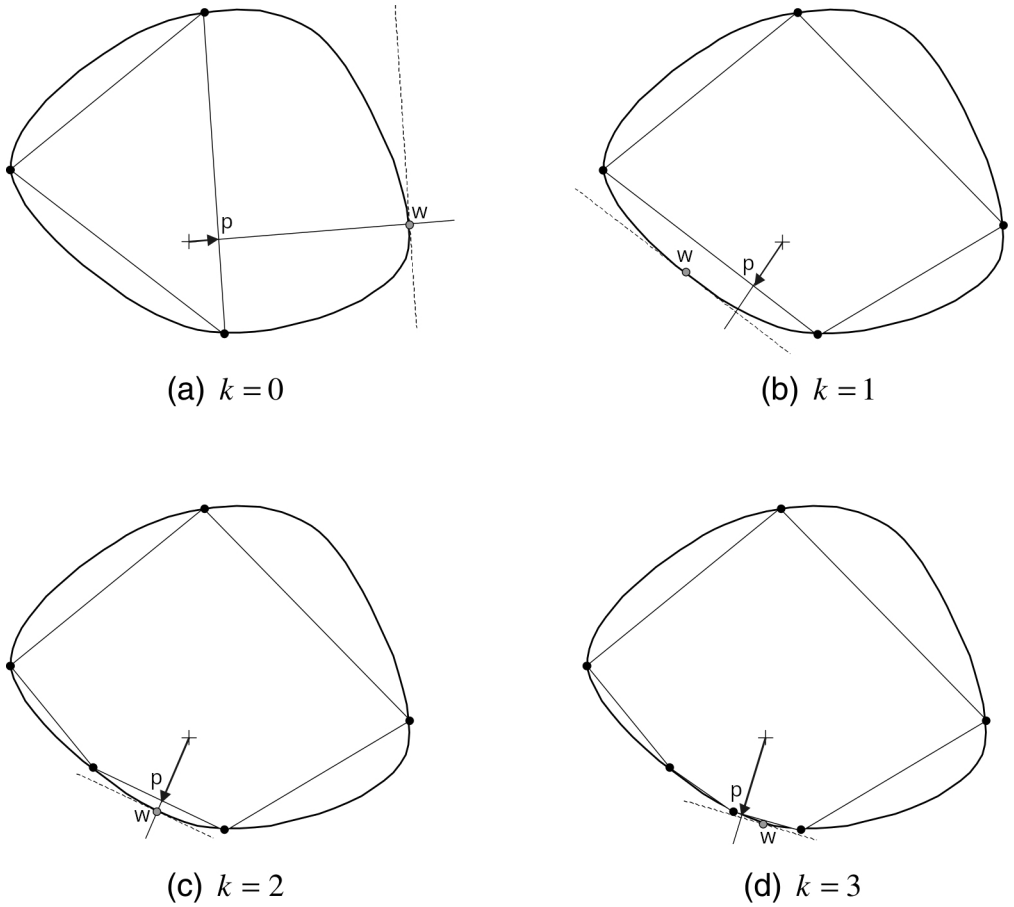


Figura 13 – Uma sequência de iterações do algoritmo de expansão de politopos. Uma seta indica o vetor mais curto $\vec{p}$. Um ponto aberto indica o novo vértice $\vec{w}$. Fonte: (BERGEN, 2001)

O Algoritmo 9 expande iterativamente o simplex final do GJK até encontrar a face da fronteira do CSO mais próxima da origem. A distância δ da origem a essa face define a profundidade de penetração, enquanto a normal $\hat{n}$ à face define a direção de separação. A tolerância ϵ (ex: 10^{-4}) e o limite de iterações são cruciais para evitar loops infinitos em casos de precisão numérica flutuante ou formas curvas aproximadas.

4.5 LIMITAÇÕES

Ao adotar métodos simplificados, abre-se mão de características essenciais de motores físicos completos. Entre as limitações mais relevantes estão:

- Ausência de conservação precisa de energia e momento, o que reduz o realismo de certas interações.
- Incapacidade de simular materiais complexos (ex.: fricção anisotrópica, torques realistas, elasticidade avançada).
- Dependência de parâmetros empíricos, sem interpretação física clara.

Algoritmo 9: EPA

Input: Simplex Q (retornado pelo GJK em caso de colisão)
Output: $\hat{n}$, δ
while *iterações* $<$ *MAX_ITER* **do**
  $e = (\vec{v}_i, \vec{v}_j) \leftarrow$ aresta mais próxima da origem
  $\hat{n}_e \leftarrow$ normal unitária externa de e
  $w \leftarrow S_{CSO}(\hat{n}_e)$
  $\delta \leftarrow \vec{w} \cdot \hat{n}_e$
  if $|\delta - d(e)| < \epsilon$ **then**
    return $\hat{n}_e, \delta$
  Remover a aresta e de Q
  Inserir as arestas $(\vec{v}_i, \vec{w})$ e $(\vec{w}, \vec{v}_j)$ em Q
return $\hat{n}_e, \delta$

- Dificuldade de lidar com sistemas altamente conectados (estruturas rígidas, máquinas, esqueletos).

Essas limitações não invalidam o uso das técnicas, mas reforçam a necessidade do uso da aplicação a cenários onde a prioridade é a responsividade, e não a precisão física.

Jittering

Em sistemas baseados em posições a estabilidade depende fortemente do processo de correção de posições. Um dos problemas mais recorrentes é o *jitter*, um tremor ou oscilação indesejada no posicionamento dos corpos, especialmente perceptível quando múltiplas restrições são aplicadas simultaneamente ou quando o sistema é altamente rígido.

Empilhamento

Métodos simplificados têm dificuldade em manter pilhas estáveis de objetos, principalmente quando as correções não são distribuídas de forma global e consistente. O empilhamento tende a "escorregar" ou colapsar devido à falta de precisão numérica adequada.

Tunneling

Ocorre quando objetos em alta velocidade atravessam outros sem detectar colisão. Métodos baseados exclusivamente em detecção discreta apresentam maior risco, especialmente quando o passo temporal é grande ou a geometria é fina.

5 OTIMIZAÇÕES

A eficiência computacional é um dos fatores determinantes para o desempenho de um motor de física em tempo real. Em jogos, animações interativas, simulações físicas e aplicações gráficas, a necessidade de atualizações contínuas e a obrigatoriedade de operar dentro de limites rigorosos de tempo tornam indispensável o uso de técnicas de otimização em todos os estágios do pipeline de simulação.

Como qualquer objeto pode potencialmente colidir com qualquer outro, uma simulação contendo n objetos requer $(n-1)+(n-2)+\cdots+1 = n(n-1)/2 = O(n^2)$ testes de pares no pior caso. Devido à complexidade quadrática, testar ingenuamente cada par torna-se impraticável mesmo para valores moderados de n.

Reduzir o custo associado ao teste de pares afetará o tempo de execução apenas linearmente. Para realmente acelerar o processo, o número de pares testados deve ser reduzido. Essa redução é realizada separando o tratamento de colisões de múltiplos objetos em duas fases: filtragem e refinamento.

Neste capítulo, descrevemos as estratégias clássicas de otimização aplicadas à detecção e resolução de colisões, com foco na divisão entre filtragem e refinamento, no uso de estruturas espaciais, na adoção de passos temporais fixos e na execução multi-thread de simulações físicas.

5.1 OBJETOS DELIMITADORES

Em sistemas de simulação física e detecção de colisões, a representação geométrica dos objetos influencia diretamente a eficiência dos cálculos. Formas complexas, com muitos vértices ou superfícies não convexas, tornam tais testes significativamente mais custosos. Uma solução comum é empregar aproximações convexas que possibilitam testes rápidos sem sacrificar excessivamente a precisão da simulação.

A principal motivação para o uso de objetos delimitadores é que formas mais simples (como caixas ou esferas) permitem testes de sobreposição muito mais baratos do que a geometria original que envolvem. Além disso, se as formas delimitadoras se intersectam, não necessariamente os objetos se intersectam; entretanto, se elas não se intersectam, necessariamente os objetos não se intersectam, como ilustrado na Figura 14. Esse fato pode ser usado para acelerar os testes de colisão, como veremos posteriormente.

Segundo Möller, Haines e Hoffman (2018), nem todos os objetos geométricos servem como objetos delimitadores eficazes. As propriedades desejáveis para objetos delimitadores incluem:

- Testes de interseção de baixo custo

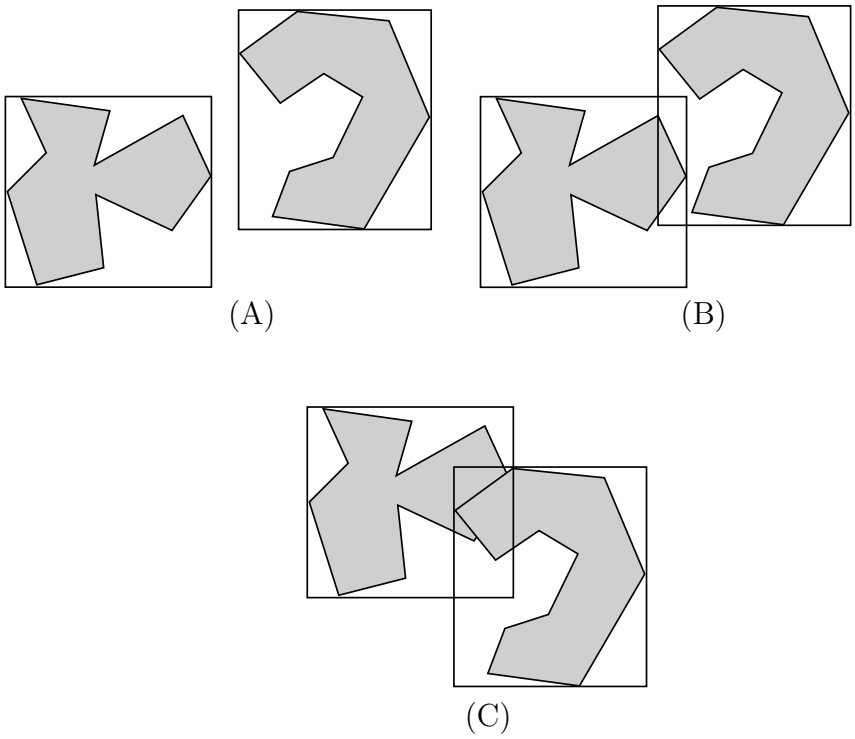


Figura 14 – Em (A) os objetos delimitadores não se intersectam, em (B) eles se intersectam porém os objetos não se intersectam, em (C) eles se intersectam e os objetos se intersectam.

- Ajuste preciso
- Cálculo econômico
- Fácil de girar e transformar
- Baixo consumo de memória

como ilustrado na Figura 15.

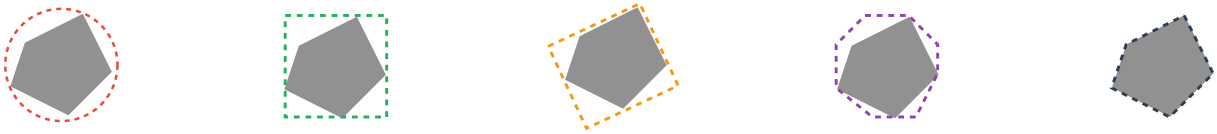


Figura 15 – Da esquerda para direita esfera delimitadora, caixa delimitadora alinhada aos eixos, caixa delimitadora orientada, K-DOP e fecho convexo.

Esfera Delimitadora

As esferas constituem o volume delimitador mais simples, definidas apenas por um centro c e um raio r:

$$\text{Sphere} = \{x \in \mathbb{R}^3 \mid \|x - c\| \leq r\}.$$

Testes de colisão entre esferas são rápidos, porém inadequados para objetos de proporções irregulares. Elipsoides oferecem melhor ajuste, mas aumentam o custo de teste. Por isso, essas formas são frequentemente utilizadas em fases preliminares da detecção, ou como nós intermediários em hierarquias de volumes delimitadores (BVH, do inglês *Bounding Volume Hierarchy*).

Caixa Delimitadora Alinhada aos Eixos Coordenados

A caixa delimitadora alinhada aos eixos (AABB, do inglês *Axis Align Bounding Box*) é um dos objetos delimitadores mais comuns. Trata-se de um paralelepípedo (ou retângulo, em 2D) cujas faces são paralelas aos eixos do sistema de coordenadas. Seu teste de interseção é simples como no Algoritmo 10.

Algoritmo 10: Teste de Intersecção AABB

Input: A e B: volumes AABB
Output: Verdadeiro se houver colisão
if $A.x_{max} < B.x_{min}$ ***ou*** $A.x_{min} > B.x_{max}$ **then**
  ∟ **return** *False*
if $A.y_{max} < B.y_{min}$ ***ou*** $A.y_{min} > B.y_{max}$ **then**
  ∟ **return** *False*
if $A.z_{max} < B.z_{min}$ ***ou*** $A.z_{min} > B.z_{max}$ **then**
  ∟ **return** *False*
return *True*

Caixa Delimitadora Orientada

Uma Caixa Delimitadora Orientada (OBB, do inglês *Oriented Bouding Box*) é uma caixa retangular que pode estar arbitrariamente rotacionada em relação aos eixos do sistema de coordenadas. A representação mais comum é feita por um ponto central c, por três vetores ortogonais $\hat{u}_i$ que compõem sua orientação e por extensões e_i, que são assumidas serem positivas.

OBBs geralmente oferecem melhor ajuste, especialmente para objetos alongados ou rotacionados, reduzindo falsos positivos. Porém, o teste de interseção é mais caro que o das AABBs e geralmente é feito usando o SAT.

Fecho Convexo

O fecho convexo de um conjunto finito de pontos P é definido como a menor região convexa que contém todos os pontos em P, sendo frequentemente utilizado como um objeto delimitador. Determinar o fecho convexo é um problema recorrente em computação geométrica, especialmente quando se deseja organizar pontos em estruturas mais simples ou acelerar operações posteriores, como testes de colisão.

Existem vários algoritmos, entre os quais o *Quickhull*, que é um algoritmo bastante conhecido para o cálculo do fecho convexo de um conjunto finito de pontos em qualquer dimensão, adotando uma estratégia de divisão e conquista semelhante ao *quicksort* (BARBER; DOBKIN; HUHDANPAA, 1996).

O Quickhull parte de um conjunto de pontos S e constrói o polígono (ou poliedro) convexo que os contém. O processo para 2 dimensões é ilustrado na Figura 16 e pode ser descrito em linhas gerais como no Algoritmo 11. Apresenta complexidade média

Algoritmo 11: Quickhull 2D

Input: Polígono Convexo
Output: Lista dos vértices do fecho convexo

1 Encontre os pontos de menor e maior coordenada em x; eles pertencem ao fecho convexo.
2 Use a linha formada pelos dois pontos para dividir o conjunto em dois subconjuntos de pontos, que serão processados de forma recursiva.
3 Para cada subconjunto, encontre o ponto mais distante da linha; ele forma um triângulo que exclui pontos interiores.
4 Repita recursivamente os dois passos anteriores nas duas linhas formadas pelos dois novos lados do triângulo.
5 O processo termina quando todos os subconjuntos estão vazios.

$O(n \log n)$ em 2D, assumindo uma distribuição uniforme dos pontos, podendo chegar a $O(n^2)$ em casos degenerados. Em 3D, adapta-se a construções poliedrais mais complexas, mantendo o mesmo princípio recursivo.

5.2 FILTRAGEM E REFINAMENTO

Em sistemas de simulação física com muitos objetos, o custo computacional da detecção de colisão pode rapidamente tornar-se inviável. Uma abordagem ingênua, baseada na verificação de todos os pares possíveis de objetos, apresenta complexidade $O(n^2)$, inviável mesmo para quantidades moderadas de entidades dinâmicas. Observa-se, entretanto, que em cenários realistas apenas uma pequena fração dos objetos encontra-se efetivamente próxima o suficiente para colidir em um dado instante. A partir dessa constatação, a detecção de colisão é tradicionalmente decomposta em duas etapas conceitualmente distintas: filtragem (*Broad Phase*) e refinamento (*Narrow Phase*).

A etapa da filtragem tem como objetivo reduzir drasticamente o número de pares candidatos à colisão por meio de testes conservadores e de baixo custo computacional. Nessa fase, não se busca determinar com exatidão se dois objetos colidem, mas apenas descartar, com segurança, aqueles pares cuja colisão é geometricamente impossível. Para isso, empregam-se objetos delimitadores simples, como caixas alinhadas aos eixos, em conjunto com estruturas de localização espacial que exploram a coerência espacial da cena.

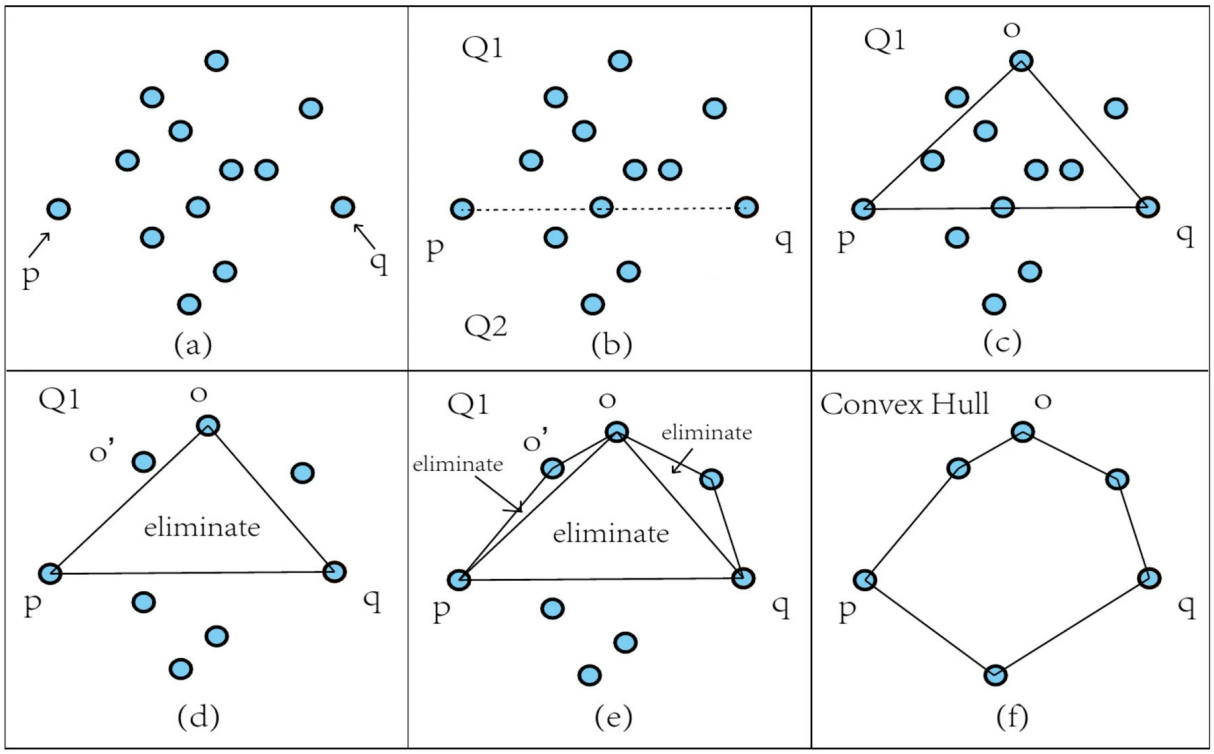


Figura 16 – Etapas do quickhull fonte: (ZENG; ZHONG; PAN, 2024).

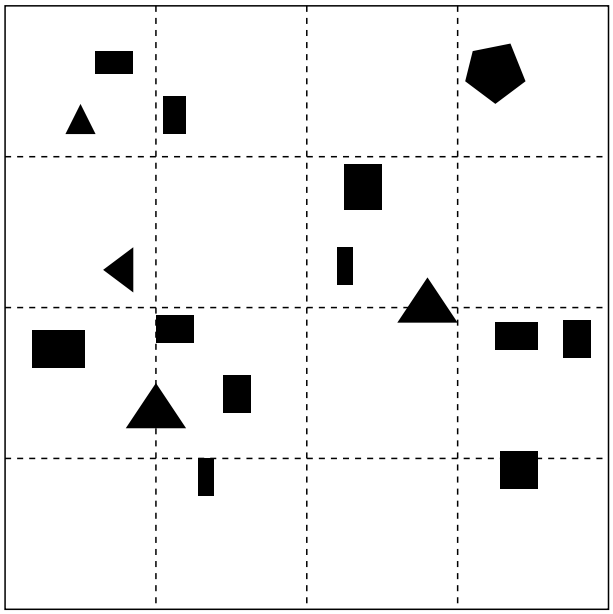
A premissa fundamental é que objetos só podem interagir se estiverem suficientemente próximos.

As técnicas de partição espacial constituem um dos principais instrumentos da etapa da filtragem. Essas técnicas subdividem o domínio contínuo da simulação em regiões menores, tipicamente convexas, cada uma associada a um conjunto de referências aos objetos que nela se encontram total ou parcialmente. Dois objetos só podem colidir se compartilharem ao menos uma dessas regiões, o que reduz significativamente o número de testes de pares necessários. Tal redução, embora dependente da distribuição espacial dos objetos, permite alcançar tempo esperado próximo de $O(n)$ ou $O(n \log n)$ em cenários bem comportados.

A literatura apresenta diversas estruturas de dados voltadas à organização espacial, incluindo abordagens hierárquicas como *Quadtrees*, *Octrees*, *Binary Space Partitioning Trees* (BSP), *K-d Trees* e *Ball Trees*. As grades uniformes (*Uniform Grids*) constituem um esquema muito usado de subdivisão espacial.

Uma grade uniforme subdivide o espaço em células de tamanho fixo e igual, associando cada objeto às células que sua caixa delimitadora intercepta, conforme ilustrado na Figura 17. Dessa forma, reduz-se o número de testes de colisão de um contra todos para apenas os pares que compartilham células. Essa abordagem assume implicitamente que a escala dos objetos é relativamente homogênea, hipótese que influencia diretamente sua eficiência.

Figura 17 – Divisão espacial em grade uniforme



Uma implementação direta da grade uniforme poderia empregar uma matriz bidimensional, na qual cada célula armazena uma lista de objetos. Contudo, essa representação apresenta limitações relevantes, como elevado consumo de memória em cenários esparsos, tamanho fixo do mundo e baixa flexibilidade para ambientes potencialmente infinitos. Para contornar essas restrições, adota-se neste trabalho uma abordagem baseada em ordenação linear das chaves de célula por varredura.

Nessa estratégia, na inicialização da grade, cada célula da grade é identificada por coordenadas inteiras (i, j), obtidas a partir das coordenadas contínuas (x, y) do objeto, segundo:

$$i = \left\lfloor \frac{x}{C} \right\rfloor, \quad j = \left\lfloor \frac{y}{C} \right\rfloor,$$

onde C representa o tamanho da célula. Como neste trabalho um objeto é representado por um conjunto de pontos, não temos acesso direto ao seu centro. Portanto, são calculadas as coordenadas de todas as células interceptadas por sua AABB, garantindo a propriedade conservadora da filtragem.

As coordenadas são linearizadas em uma chave única da célula (*cell key*) através de um mapeamento linear simples:

$$Key_{cell} = i + j \times ncol$$

onde $ncol$ é o número dec colunas da grade. Os pares $(Key_{cell}, Index_{obj})$ são salvos num array, onde $Index_{obj}$ representa o índice do objeto no array original ou referência ao objeto na memória.

Uma vez construída a grade, a filtragem consiste em ordenar os pares pela chave da célula. Nessa etapa pode-se utilizar diversos algoritmos de ordenação encontrados na literatura como o *Radix Sort*. Neste trabalho será usado o algoritmo de ordenação nativo

do navegador que depende da implementação do interpretador javascript, como TimSort para o V8 (Google Chrome) ou o MergeSort para o SpiderMonkey (Mozilla Firefox).

Após a ordenação, os objetos que compartilham a mesma célula aparecem em posições contíguas no array, o quê favorece a localidade espacial e temporal, menos *cache misses* e *page faults*. A partir dessa organização, é possível gerar os pares candidatos à colisão percorrendo o array ordenado e agrupando os objetos por célula. Para evitar testes redundantes, é necessário eliminar dois tipos de duplicatas: (1) se dois objetos A e B caem na mesma célula, é preciso testar apenas o par (A, B) e não repetir para o par (B, A); de forma análoga para (2) se um objeto é grande suficiente para interceptar múltiplas células. Essa etapa é essencial para preservar os ganhos de desempenho proporcionados pela filtragem espacial.

Essa abordagem permite armazenar apenas as células efetivamente ocupadas, reduzindo o consumo de memória e permitindo a simulação em espaços conceitualmente ilimitados.

Em ambientes dinâmicos, nos quais objetos se movem continuamente e podem ser criados ou removidos, a manutenção incremental da estrutura espacial torna-se um aspecto crítico. Embora seja possível atualizar apenas os elementos afetados por cada objeto em movimento, essa abordagem exige controle adicional e estruturas auxiliares. Neste trabalho, opta-se deliberadamente por uma estratégia mais simples: a estrutura é reutilizada a cada passo de tempo a partir de um array linear, cujo contador de elementos é zerado e preenchido novamente com base na ordenação das chaves de célula. Essa decisão representa um trade-off arquitetural, privilegiando simplicidade e previsibilidade de execução em detrimento de possíveis otimizações incrementais.

O desempenho de métodos baseados em grade uniforme é fortemente influenciado pela escolha do tamanho da célula. Idealmente, cada objeto deveria ocupar aproximadamente uma única célula; nesse caso, em duas dimensões, um objeto pode interceptar no máximo quatro células adjacentes. Segundo Ericson (2004b), escolhas inadequadas para esse parâmetro podem degradar significativamente o desempenho, destacando-se quatro fatores principais:

1. células excessivamente pequenas aumentam o custo de atualização da estrutura;
2. células grandes demais agrupam muitos objetos, reduzindo a eficácia da filtragem;
3. objetos de geometria complexa podem exigir subdivisão adicional;
4. cenários com grande variação de escala demandam abordagens hierárquicas ou híbridas.

Assim, a adoção de uma grade uniforme não constitui uma solução universal, mas uma decisão arquitetural dependente das características da cena simulada.

Ao término da filtragem, obtém-se um conjunto P de pares candidatos a colisão, com redução significativa do custo computacional em relação ao método quadrático. Ainda assim, no pior caso, quando muitos objetos se concentram em poucas células, a complexidade pode degradar novamente para $O(n^2)$, evidenciando o caráter heurístico dessa etapa.

A etapa de refinamento realiza testes geométricos precisos nos pares de candidatos em P. Essa fase é chamada de Narrow Phase. Utiliza-se algoritmos mais sofisticados para determinar se os objetos estão de fato colidindo, como SAT, GJK e EPA, vistos anteriormente.

5.3 COMO DETERMINAR O INTERVALO DE INTEGRAÇÃO DA FÍSICA

Em simulações físicas, é necessário definir um passo de tempo, usualmente denotado por dt (*Delta Time*), que determina o intervalo entre duas integrações consecutivas do sistema. A escolha desse parâmetro é crítica, pois influencia diretamente tanto a precisão quanto o custo computacional da simulação.

De modo geral, quanto menor o valor de dt, mais suave e mais fiel à realidade tende a ser a simulação. Isso ocorre porque o sistema é amostrado com maior frequência, reduzindo erros numéricos e melhor capturando eventos importantes, como colisões. Por outro lado, valores muito pequenos de dt aumentam significativamente o custo computacional, uma vez que, a cada passo, é necessário executar rotinas potencialmente caras, como detecção e resposta a colisões, cujo custo cresce com o número de objetos na cena.

Em cenários com restrições de desempenho, por exemplo, aplicações em tempo real ou com grande quantidade de objetos, pode não ser viável utilizar um dt muito pequeno. Nesses casos, opta-se por um dt maior, reduzindo o número de iterações da simulação por unidade de tempo. Entretanto, essa escolha introduz erros mais significativos. Um problema clássico associado a valores elevados de dt é o fenômeno conhecido como *tunneling*, no qual um objeto pode atravessar outro sem que a colisão seja detectada, pois o instante de contato não é amostrado durante a integração.

Além disso, é importante distinguir entre o passo de tempo da simulação física e o passo de tempo da renderização. Em muitos sistemas, esses dois processos operam com frequências diferentes: a simulação física pode utilizar um dt fixo e relativamente pequeno para garantir estabilidade e reprodutibilidade, enquanto a renderização ocorre com base na taxa de atualização do sistema (frames por segundo), que pode variar dinamicamente. Nesses casos, técnicas de interpolação são frequentemente empregadas para garantir uma visualização suave, mesmo quando a simulação física é executada em uma frequência diferente da renderização.

No contexto da integração numérica, existem dois esquemas principais. O primeiro é a integração com passo fixo, na qual o valor de dt permanece constante ao longo da

simulação. Essa abordagem é amplamente utilizada por sua previsibilidade e reprodutibilidade, sendo especialmente importante em aplicações determinísticas, como simulações físicas replicáveis. O segundo esquema é a integração com passo variável, em que o valor de dt é ajustado dinamicamente com base no tempo decorrido entre quadros. Embora essa abordagem seja comum em aplicações interativas, ela pode introduzir inconsistências na simulação, uma vez que diferentes execuções, ou diferentes máquinas, podem produzir resultados distintos devido à variação no número de passos de integração.

Em síntese, a escolha de dt representa um compromisso entre precisão e desempenho. Idealmente, deseja-se utilizar o menor valor possível para garantir estabilidade e exatidão, mas restrições computacionais frequentemente impõem a necessidade de valores maiores, exigindo cuidados adicionais para mitigar erros numéricos e artefatos na simulação.

5.4 EXECUÇÃO PARALELA E PARALELISMO DE DADOS NA SIMULAÇÃO FÍSICA

A simulação física em aplicações interativas apresenta elevado custo computacional, especialmente quando envolve integração temporal, detecção de colisões e resolução iterativa de restrições. No contexto da Web, esse custo torna-se ainda mais relevante devido às limitações da thread principal, responsável não apenas pela execução do código da aplicação, mas também pela renderização gráfica, tratamento de eventos de entrada e atualização da interface do usuário. Dessa forma, torna-se necessário discutir estratégias de paralelismo que permitam manter a responsividade da aplicação sem comprometer a estabilidade e a previsibilidade da simulação física.

Uma abordagem amplamente adotada consiste na separação conceitual entre as rotinas de renderização e simulação física. Enquanto a renderização é executada na thread principal, geralmente sincronizada com o ciclo de atualização visual por meio de mecanismos como *requestAnimationFrame*, a simulação física pode ser executada em uma thread separada, utilizando *Web Workers*. Essa separação reduz a probabilidade de bloqueios na interface e permite que o cálculo físico seja realizado de forma assíncrona em relação ao processo de desenho.

Neste trabalho foi implementando um modelo de separação da thread de renderização e simulação física para o artefato web. A thread responsável pela simulação apenas escreve o estado físico mais atual e a renderização consome apenas o estado físico mais recente disponível, como ilustrado na Figura 18.

A simulação física apresenta dependências de dados que dificultam sua paralelização completa. Na resolução de colisões e restrições, múltiplas interações podem envolver um mesmo corpo rígido ou partícula. Por exemplo, colisões entre os pares (A, B) e (B, C) compartilham o corpo B, o que exige coordenação cuidadosa para evitar inconsistências. No método de Jakobsen (2001), em particular, a ordem de aplicação das restrições influ-

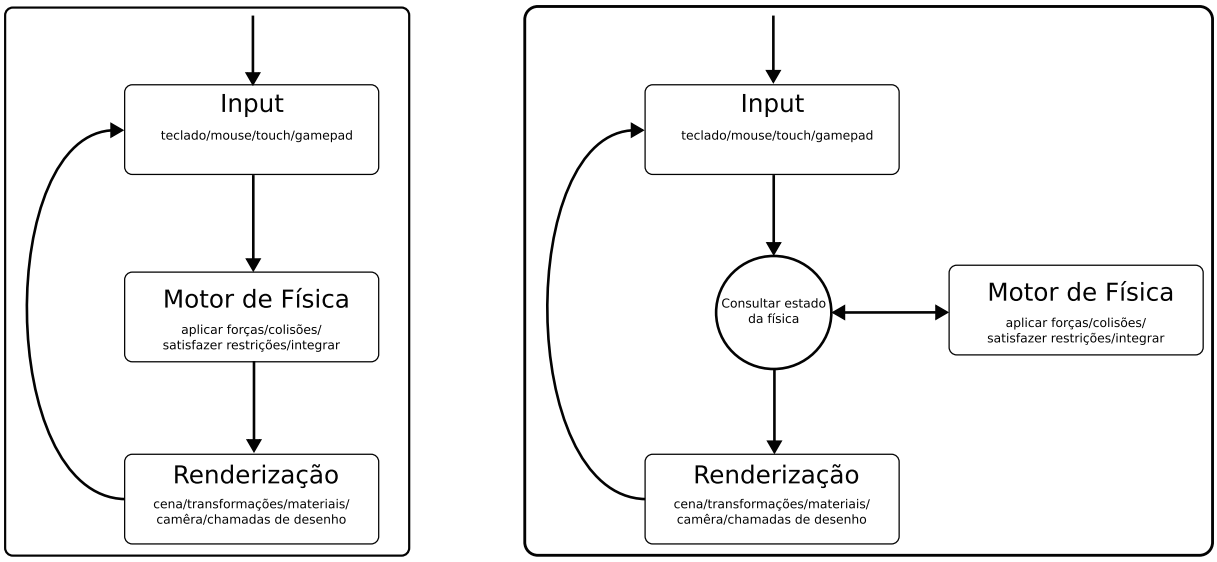


Figura 18 – À esquerda esquema tradicional single-thread. À direita esquema com simulação rodando numa thread dedicada a passo fixo.

encia o resultado final, tornando a execução paralela ainda mais sensível a variações na ordem de processamento.

Além disso, um dos principais problemas está relacionado à distribuição de carga (*load balancing*). Em simulações físicas, o custo computacional não é uniforme: regiões do espaço com maior densidade de corpos exigem mais testes de colisão e mais iterações de resolução de restrições, enquanto regiões esparsas demandam pouco processamento. Como essa distribuição varia dinamicamente ao longo do tempo, torna-se difícil dividir o trabalho de forma equilibrada entre várias threads.

Outro desafio fundamental refere-se à consistência dos dados. Em um ambiente multithread, diferentes threads podem acessar e modificar simultaneamente os mesmos estados físicos, como posições e orientações dos corpos. Essa situação pode levar a condições de corrida, resultados inconsistentes e instabilidade numérica. Em simulações físicas, utilizar estados parcialmente atualizados é particularmente problemático, pois pequenas inconsistências podem se propagar e amplificar ao longo das iterações.

A necessidade de mecanismos de sincronização, como travas (*locks*), operações atômicas e barreiras, introduz sobrecarga adicional e pode reduzir significativamente os ganhos esperados com o paralelismo. Em muitos casos, o custo de sincronização supera o benefício obtido com a execução paralela, especialmente quando a granularidade das tarefas é fina, como na resolução individual de colisões.

Outro aspecto relevante é o não-determinismo introduzido pela execução concorrente. A ordem de escalonamento das threads pode variar entre execuções, levando a resultados físicos ligeiramente diferentes mesmo sob condições iniciais idênticas. Esse comportamento dificulta a depuração, a reprodução de simulações e a implementação de funcionalidades como *replays* ou sincronização em aplicações distribuídas.

Por essas razões, muitas engines físicas modernas optam por paralelizar apenas etapas bem definidas e fracamente acopladas da simulação, como a detecção de colisão na fase da filtragem, a resolução de colisões na fase de refinamento e a resposta a colisão em um único thread lógico.

Nem todo paralelismo está associado à execução em múltiplas threads. O paralelismo de dados, explorado por meio de instruções SIMD (*Single Instruction, Multiple Data*), permite aplicar a mesma operação a múltiplos elementos simultaneamente dentro de um único fluxo de execução. Esse modelo é particularmente adequado para etapas da simulação física que envolvem operações homogêneas sobre grandes conjuntos de dados, como a integração temporal das posições, a atualização de velocidades implícitas e testes simples de colisão.

Ao contrário do paralelismo baseado em threads, SIMD não introduz condições de corrida nem exige mecanismos complexos de sincronização, uma vez que todas as operações ocorrem de forma determinística dentro de um único contexto de execução. No contexto da Web, o suporte a SIMD em WebAssembly permite explorar esse tipo de paralelismo de forma eficiente, desde que os dados estejam organizados em layouts de memória apropriados, como *Structure of Arrays* (SoA).

O uso de WebAssembly (WASM) oferece vantagens significativas para a implementação de simulações físicas intensivas. Por possuir tipagem estática, memória linear e maior previsibilidade de desempenho, WASM é mais adequado para laços computacionais intensivos do que JavaScript tradicional. Quando combinado com SIMD, um único thread de execução em WASM pode atingir desempenho comparável ou superior a implementações JavaScript multithread, sem incorrer nos problemas de sincronização e não-determinismo associados à concorrência.

Essa abordagem justifica arquiteturas nas quais a simulação física completa é executada em um único worker utilizando WASM, enquanto a thread principal permanece responsável pela renderização e interação com o usuário. Dessa forma, o paralelismo é explorado verticalmente, por meio de instruções vetoriais e melhor uso do hardware subjacente, em vez de horizontalmente, por meio da criação de múltiplas threads concorrentes.

O WebGPU introduz a possibilidade de explorar paralelismo massivo por meio da GPU, sendo particularmente adequado para tarefas altamente paralelizáveis e com baixo acoplamento entre dados. Na simulação física, isso inclui etapas como a detecção de colisão em fase da filtragem, a construção de grades uniformes e a geração de pares candidatos à colisão. Essas operações envolvem grandes volumes de dados independentes e se beneficiam diretamente do modelo de execução da GPU.

Entretanto, a resolução iterativa de restrições físicas, especialmente em métodos como o de Jakobsen (2001), apresenta dependências complexas que limitam sua adequação à execução na GPU. Assim, o uso de WebGPU deve ser encarado como complementar, e não como substituto direto, da simulação física tradicional executada na CPU.

Com base nas considerações discutidas, uma arquitetura híbrida mostra-se mais adequada para simulações físicas simplificadas na Web. Nessa arquitetura, a renderização é realizada na *main thread*, a simulação física principal é executada em um worker dedicado utilizando WebAssembly, e o paralelismo de dados é explorado por meio de SIMD. Opcionalmente, etapas específicas e fracamente acopladas, como a fase ampla da detecção de colisões, podem ser delegadas à GPU via WebGPU.

Essa abordagem reduz a complexidade associada à programação concorrente, melhora a previsibilidade dos resultados e mantém um equilíbrio entre desempenho e robustez, sendo particularmente adequada para aplicações Web interativas e educacionais.

6 IMPLEMENTAÇÃO E AVALIAÇÃO EXPERIMENTAL

Este capítulo apresenta a implementação do sistema desenvolvido e os experimentos conduzidos com o objetivo de avaliar seu desempenho, escalabilidade e corretude geométrica. Em particular, busca-se analisar o impacto de diferentes estratégias nas etapas de filtragem e refinamento, bem como comparar os algoritmos SAT e GJK/EPA em um ambiente de execução web, no qual restrições de desempenho e responsividade são relevantes.

Os experimentos foram realizados em um computador com as seguintes especificações:

- Processador: AMD Ryzen 5 5500U
- Placa de Vídeo: AMD Radeon Vega 7 (integrada)
- Memória RAM: 24 GB DDR4
- Sistema Operacional: Ubuntu 24.04 LTS

6.1 ARQUITETURA E IMPLEMENTAÇÃO

Foi desenvolvida uma biblioteca em *TypeScript* responsável pela implementação dos algoritmos de detecção de colisão, sendo reutilizada tanto na aplicação de visualização quanto no ambiente de testes. A engine permite a execução em dois modos distintos: utilizando o algoritmo do Teorema do Eixo Separador (SAT) ou a combinação dos algoritmos GJK/EPA.

A arquitetura do sistema foi projetada de forma modular, separando a lógica geométrica, os algoritmos de detecção e resposta à colisão e a camada de visualização, delegando esta última à aplicação cliente. Essa separação favorece a reutilização de código e possibilita a comparação direta entre diferentes abordagens de detecção de colisão.

Os objetos convexos são representados como listas de vértices ordenados, associadas a um conjunto de restrições. Cada restrição atua sobre um par de partículas, aplicando um processo de relaxamento que permite modelar tanto estruturas rígidas quanto deformáveis. Essa modelagem possibilita a integração da detecção de colisão com simulações físicas baseadas em partículas.

Este trabalho disponibiliza uma implementação pública com o objetivo de permitir a reprodução dos experimentos e a extensão das técnicas apresentadas [1].

[1] Disponível em: https://github.com/devdiegomatos/simplified-physics-animation-web

6.2 AMBIENTE DE VISUALIZAÇÃO

Com o objetivo de permitir a análise qualitativa do comportamento dos algoritmos, foram desenvolvidas duas aplicações interativas (*demos*) [2]. utilizando *Vue.js*, *Vite* e *p5.js*

A primeira demonstração consiste em um ambiente contendo múltiplos objetos convexos, com número de vértices variando entre 3 e 8, inseridos em uma caixa com obstáculos. Esse cenário permite a observação da dinâmica de colisões em tempo real. A interface possibilita o controle de parâmetros relevantes, como a quantidade de objetos, o algoritmo de detecção de colisão (SAT ou GJK/EPA) e a estratégia de filtragem, incluindo o uso de uma grade uniforme. A Figura 19 ilustra esse ambiente.

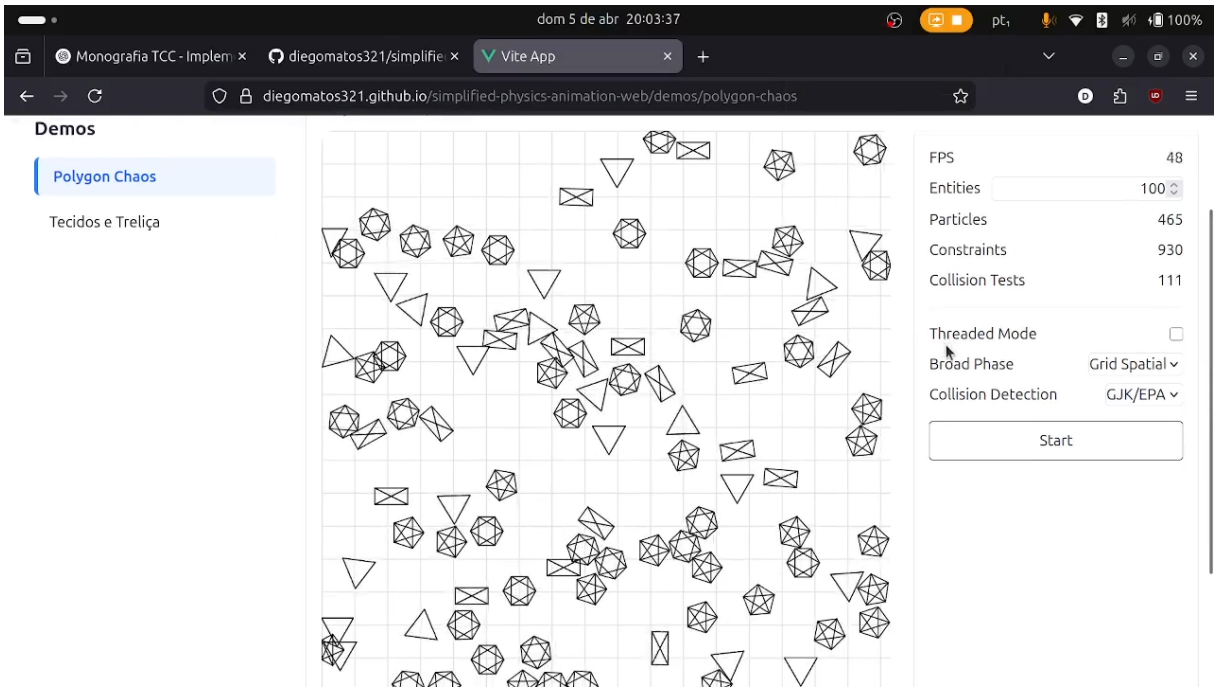


Figura 19 – Ambiente de simulação com múltiplos objetos convexos em uma caixa, permitindo a configuração de parâmetros de colisão e filtragem. Fonte: Autor.

A segunda demonstração apresenta simulações de tecidos e treliças, evidenciando a interação entre objetos deformáveis. Nesse cenário, as restrições entre partículas são responsáveis pela manutenção estrutural, enquanto o sistema de colisão garante a interação física entre os elementos. A Figura 20 apresenta um exemplo desse ambiente.

6.3 METODOLOGIA DE TESTES

Os experimentos foram conduzidos com três objetivos principais:

- Avaliar o impacto da utilização de uma estrutura de filtragem no desempenho do sistema;

[2] https://devdiegomatos.github.io/simplified-physics-animation-web/

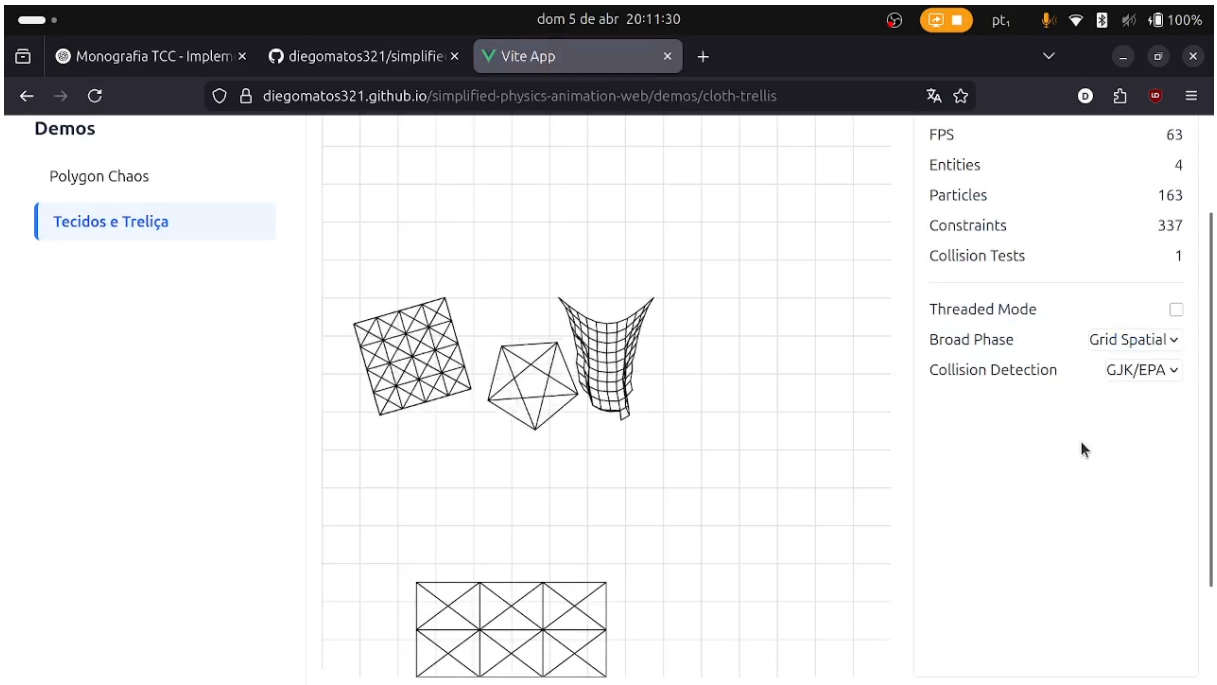


Figura 20 – Simulação de estruturas deformáveis, como tecidos e treliças, utilizando restrições entre partículas e detecção de colisão. Fonte: Autor.

- Comparar o desempenho dos algoritmos SAT e GJK/EPA na fase de refinamento;
- Analisar a escalabilidade das abordagens em função do número de objetos.
- Comparar o desempenho da implementação desenvolvida com uma biblioteca comumente usada em jogos. Aqui, usamos a biblioteca *Planck.js* (**??**).

Os testes foram realizados sem a etapa de visualização, a fim de isolar o custo computacional da detecção de colisão. Para garantir a reprodutibilidade dos experimentos, foi utilizada geração pseudoaleatória com semente fixa por meio da biblioteca *seedrandom* (**??**), uma vez que a função *Math.random* não permite controle de semente.

O cenário de teste corresponde ao ambiente apresentado na Figura 19, no qual objetos convexos são posicionados aleatoriamente dentro de uma região delimitada. Foram variados o número de objetos e o tamanho do espaço, de modo a manter aproximadamente constante a densidade de ocupação. Essa estratégia permite avaliar o impacto do crescimento do sistema sem introduzir vieses relacionados à distribuição espacial.

Foram considerados quatro casos experimentais:

- SAT sem fase de filtragem;
- SAT com grade uniforme na fase de filtragem;
- GJK/EPA sem fase de filtragem;
- GJK/EPA com grade uniforme na fase de filtragem.

O número de objetos foi variado de 1000 a 10000, em incrementos de 1000. Para cada configuração, foram realizadas 180 execuções independentes, sendo computadas as médias das métricas coletadas.

As métricas analisadas incluem o tempo médio de execução, o número de testes de colisão realizados e o número de colisões detectadas, permitindo avaliar tanto o custo computacional quanto a corretude dos algoritmos. Essa abordagem permite a visualização simultânea de todas as configurações, preservando a comparabilidade entre os métodos.

A biblioteca *Planck.js* é um motor de física 2D para JavaScript, reescrito a partir do Box2D, amplamente utilizado no desenvolvimento de jogos digitais. A *Planck.js* emprega uma estrutura de árvore hierárquica dinâmica de volumes delimitadores (AABB) na fase de filtragem, enquanto a detecção de colisão na fase de refinamento baseia-se em algoritmos como GJK e EPA.

Para garantir a comparabilidade dos resultados, foram utilizados cenários equivalentes aos descritos anteriormente, mantendo-se consistentes os parâmetros de número de objetos, distribuição espacial e número de execuções. Devido a forma como a biblioteca *Plank*é implementada, foi analisada apenas a métrica de tempo médio de execução.

6.4 RESULTADOS EXPERIMENTAIS

Os resultados obtidos permitem analisar o comportamento dos algoritmos sob diferentes configurações e validar empiricamente as hipóteses estabelecidas.

A Figura 21 evidencia o impacto direto da etapa da filtragem na redução do número de testes de colisão. A utilização da grade uniforme reduz drasticamente a quantidade de pares avaliados, refletindo diretamente na melhoria de desempenho observada. Devido à diferença de ordens de grandeza entre os cenários com e sem fase de filtragem, os resultados são apresentados em escala logarítmica no eixo vertical.

A Figura 22 apresenta o tempo médio de execução em função do número de objetos para as diferentes configurações avaliadas. Observa-se que os cenários sem utilização de grade uniforme apresentam crescimento acentuado, compatível com comportamento quadrático, enquanto as abordagens com filtragem demonstram crescimento significativamente mais lento.

Nota-se que as curvas correspondentes aos algoritmos SAT e GJK/EPA apresentam comportamento semelhante. Embora o tempo médio do SAT seja levemente inferior ao GJK/EPA, no geral a escolha da estratégia de filtragem possui maior impacto no desempenho global do sistema do que o algoritmo utilizado na fase de refinamento.

Observa-se também que, em cenários de alta densidade, o custo computacional cresce rapidamente na ausência de estruturas de filtragem, evidenciando a importância de técnicas de particionamento espacial para aplicações em tempo real.

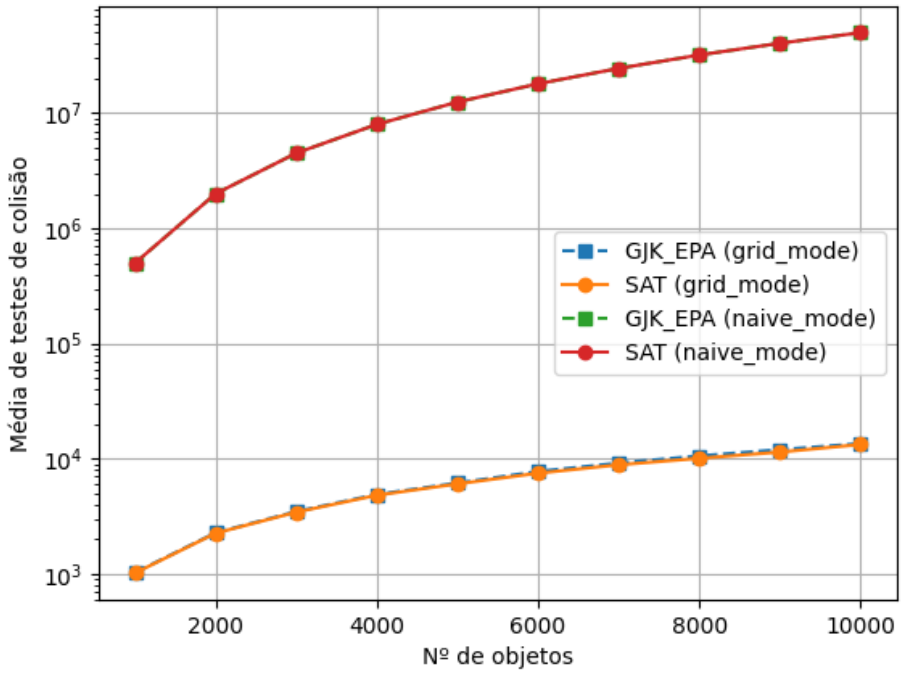


Figura 21 – Número médio de testes de colisão em função do número de objetos para diferentes configurações: SAT e GJK/EPA, com e sem utilização de grade uniforme. Fonte: Autor.

A Figura 23 apresenta o tempo médio de execução em função do número de objetos para a biblioteca *Planck.js*. Observa-se que o desempenho da *Planck.js* apresenta comportamento consistente com o uso de uma estrutura eficiente na fase de filtragem, evidenciando crescimento controlado mesmo com o aumento do número de objetos. Esse resultado confirma a eficácia do uso de árvores hierárquicas dinâmicas na redução do número de pares de objetos avaliados.

De forma geral, os resultados indicam que a combinação de algoritmos eficientes na fase de *narrow-phase* com estratégias adequadas de *broad-phase* é essencial para garantir desempenho e escalabilidade em sistemas de detecção de colisão.

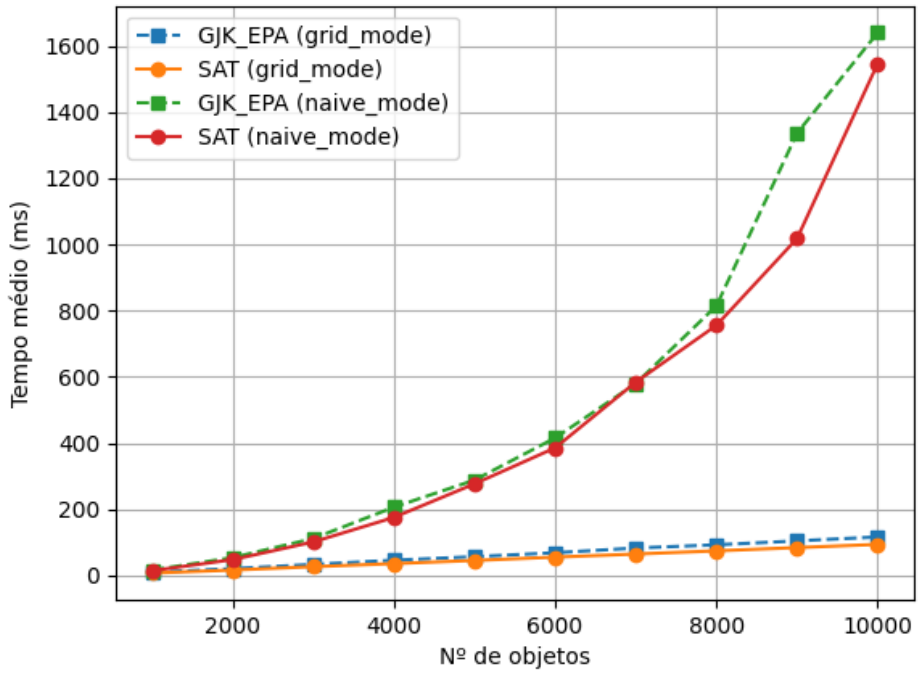


Figura 22 – Tempo médio de execução em função do número de objetos para diferentes configurações: SAT e GJK/EPA, com e sem utilização de grade uniforme. Fonte: Autor.

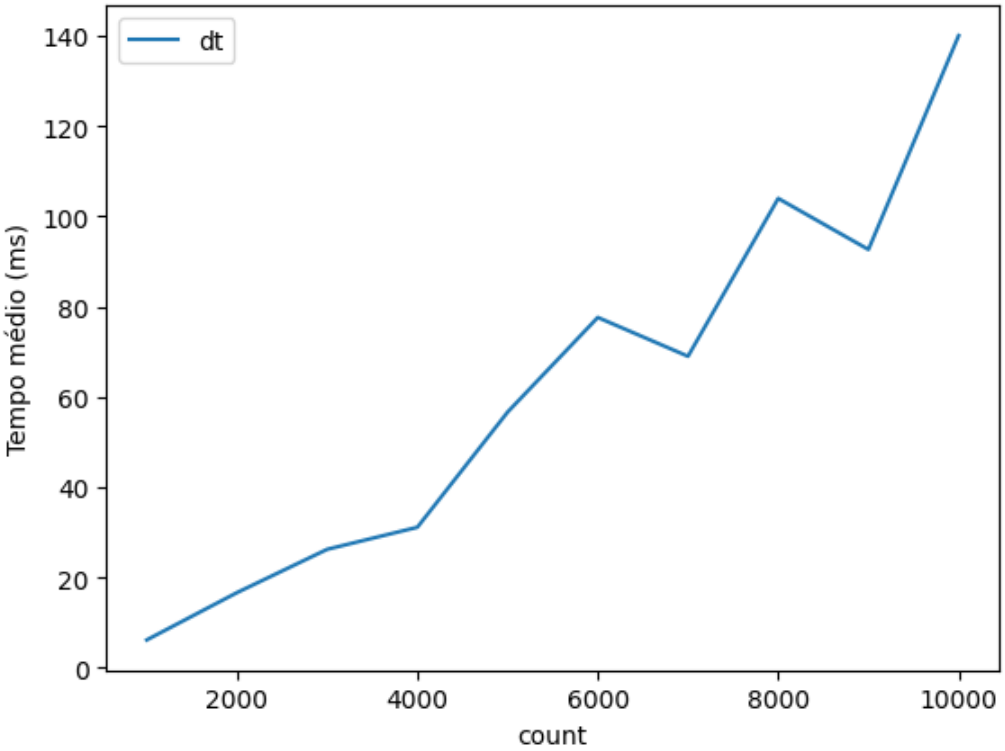


Figura 23 – Tempo médio de execução em função do número de objetos com a biblioteca *Planck.js*. Fonte: Autor.

7 CONCLUSÕES

Os experimentos realizados confirmam a viabilidade de um sistema de animação física simplificada, eficiente e estável, totalmente implementado em ambiente web. A combinação entre o integrador de Verlet, as otimizações de detecção de colisão e a execução multi-threaded proporcionou resultados consistentes e visualmente plausíveis em tempo real.

Tais resultados demonstram que é possível construir um motor físico leve e acessível, adequado para aplicações educacionais, jogos independentes e simulações científicas interativas, sem a necessidade de bibliotecas externas ou dependências proprietárias.

Como trabalhos futuros, propõe-se:

- Realizar testes em ambientes 3D
- Implementar hierarquias de volumes limitadores (BVH);
- Migrar a execução paralela para WebGPU Compute Shaders, permitindo simulação massiva em GPU.

REFERÊNCIAS

AZEVEDO, A. C. E. **Computação gráfica - Teoria e prática**. [S.l.]: Editora Campus, Ltda, 2003.

BARBER, C. B.; DOBKIN, D. P.; HUHDANPAA, H. The quickhull algorithm for convex hulls. **ACM Transactions on Mathematical Software (TOMS)**, Acm New York, NY, USA, v. 22, n. 4, p. 469–483, 1996.

BERGEN, G. v. d. Proximity queries and penetration depth computation on 3d game objects. 01 2001.

CATTO, E. **Box2D: A 2D Physics Engine for Games**. 2011. Acesso em: nov. 2025. Disponível em: https://box2d.org.

COUMANS, E. **Bullet Physics Simulation**. 2015. Acesso em: nov. 2025. Disponível em: https://pybullet.org.

dyn4j. **SAT (Separating Axis Theorem)**. 2010. https://dyn4j.org/2010/01/sat. Acesso em: nov. de 2025.

ERICSON, C. **The Gilbert-Johnson-Keerthi (GJK) Algorithm**. 2004. SIGGRAPH Presentation.

ERICSON, C. **Real-time collision detection**. [S.l.]: Crc Press, 2004.

GILBERT, E.; FOO, C.-P. Computing the distance between general convex objects in three-dimensional space. **IEEE Transactions on Robotics and Automation**, v. 6, n. 1, p. 53–61, 1990.

GILBERT, E.; JOHNSON, D.; KEERTHI, S. A fast procedure for computing the distance between complex objects in three-dimensional space. **IEEE Journal on Robotics and Automation**, v. 4, n. 2, p. 193–203, 1988.

Havok. **Havok Physics**. 2024. Acesso em: nov. 2025. Disponível em: https://www.havok.com/havok-physics/.

JAKOBSEN, T. Advanced character physics. In: **Proceedings of the Game Developer's Conference 2001**. San Jose, CA: Game Developers Conference, 2001. Presented at Game Developer's Conference 2001.

LINAHAN, J. **A Geometric Interpretation of the Boolean Gilbert-Johnson-Keerthi Algorithm**. 2015. Disponível em: https://arxiv.org/abs/1505.07873.

MÖLLER, T.; HAINES, E.; HOFFMAN, N. **Real-time rendering**. 4. ed. [S.l.]: CRC Press, 2018.

MÜLLER, M. et al. Position based dynamics. **Journal of Visual Communication and Image Representation**, v. 18, n. 2, p. 109–118, 2007. ISSN 1047-3203. Disponível em: https://www.sciencedirect.com/science/article/pii/S1047320307000065.

MURATORI, C. **Implementing GJK (2006)**. 2006. https://caseymuratori.com/blog_0003. Acessado em: 12 de Outubro de 2025.

NVIDIA Omniverse. **PhysX SDK**. https://github.com/NVIDIA-Omniverse/PhysX. BSD-3 license, Acesso em: nov. 2025.

ROCKAFELLAR, R. T. **Convex Analysis**. Princeton, NJ: Princeton University Press, 1970. (Princeton Landmarks in Mathematics and Physics).

Wikipedia contributors. **Carathéodory's theorem (convex hull) — Wikipedia, The Free Encyclopedia**. 2025. https://en.wikipedia.org/w/index.php?title=Carath%C3%A9odory%27s_theorem_(convex_hull)&oldid=1313647530. [Online; accessed 18-February-2026].

ZENG, T.; ZHONG, C.; PAN, T. Clustering explanation based on multi-hyperrectangle. **Scientific Reports**, v. 14, 12 2024.